

The University of Faisalabad

Lab Manual

**DS-303
Data Mining**

Submitted by

Muhammad Zain Sattar 2023-BS-AI-052(A)

Submitted to

Mr. Muhammad Arham

Department of Computer Science

TABLE OF CONTENTS

Lab 01	KNIME Introduction.....	4
Lab 02	Customer Churn Prediction.....	10
Lab 03	Data Cleaning.....	15
Lab 04	Data Preprocessing.....	19
Lab 05	Market Basket Analysis.....	26
Lab 06	Apriori Algorithm — Product Combination Analysis.....	30
Lab 07	FP-Growth — Efficient Frequent Pattern Mining.....	33
Lab 08	Python Basics — Reusable Data Processing Module.....	36
Lab 09	Decision Tree.....	40
Lab 10	Naïve Bayes & KNN.....	43
Lab 11	SVM — Support Vector Machine.....	47
Lab 12	Clustering — K-Means.....	51
Lab 13	Outlier Detection.....	55
Lab 14	Scraping & Sentiment Analysis.....	58
Lab 15	Final Capstone Project.....	60

Lab 1: KNIME Introduction

Real-World Scenario: A retail company stores large volumes of transaction records across product categories. Without organized analysis, raw data cannot support strategic decision-making. Business managers require clear analytical outputs to measure overall revenue, estimate average customer purchase value, identify top-earning categories, and determine which products yield the highest profit contribution.

Dataset Description

The dataset applied in this exercise is a **Retail Sales CSV** file containing detailed transaction records. Each row represents one completed sale with fields including Transaction ID, Product Title, Product Category, Quantity Sold, Unit Price, Purchase Date, and Sales Amount.

Software Tool: KNIME Analytics Platform

KNIME is a visual analytics platform that allows users to create data workflows by connecting functional nodes. Every node handles a specific task — importing files, cleaning data, applying formulas, generating summaries, or performing aggregations. Combined, these nodes form a complete data analysis pipeline.

Learning Outcomes

- 1 Load a CSV sales dataset into KNIME using the CSV Reader node.
- 2 Inspect and adjust column data types for accurate processing.
- 3 Generate descriptive statistics to understand dataset properties.
- 4 Create a new Revenue column using the Math Formula node.
- 5 Analyse four major retail KPIs using GroupBy and Sorter nodes.

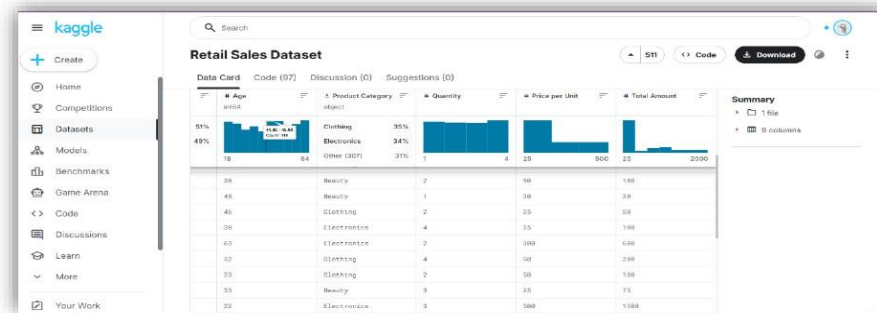
Procedure

STEP 1 Download the Dataset

Obtain the Retail Sales dataset in CSV format from the provided source and save it locally for analysis.

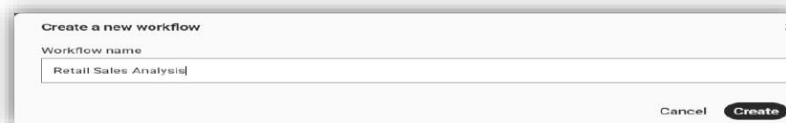
STEP 2 Create a New Workflow

Launch KNIME Analytics Platform and create a new workflow named *Retail Sales Analysis* to organise the analytical process.



Step 2: Create a New Workflow

Launch KNIME Analytics Platform and create a new workflow named *Retail Sales Analysis* to organize the analytical process.



5

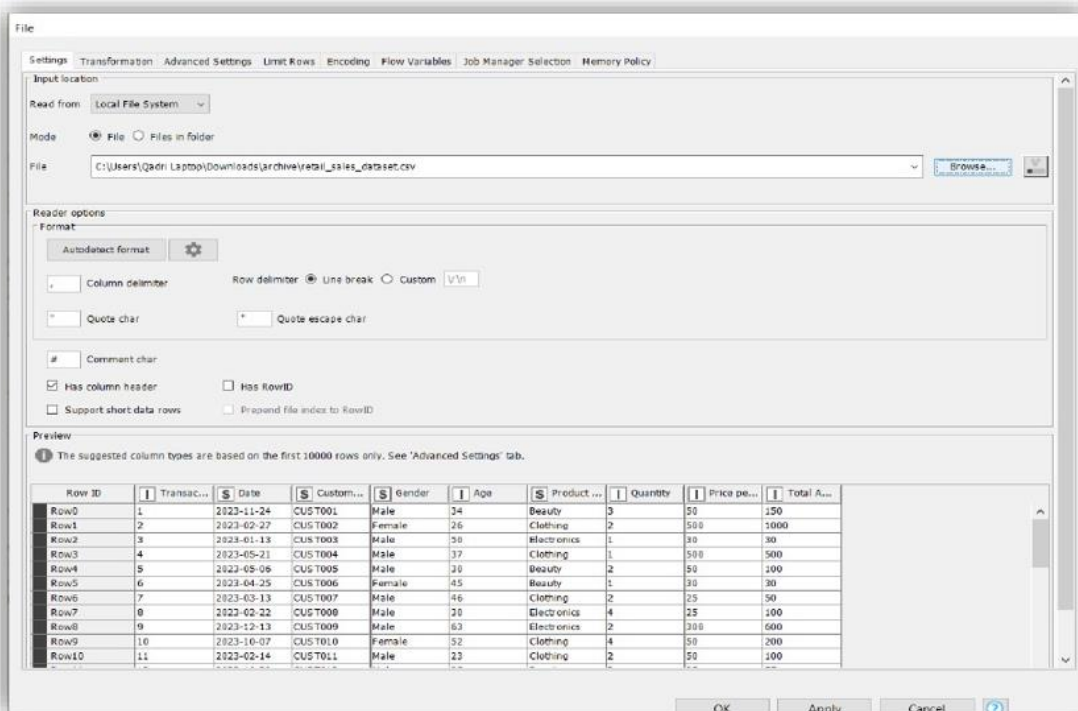
Figure 1.1 — Kaggle dataset preview & KNIME workflow creation dialog

STEP 3 Import the Dataset

Use the CSV Reader node to load the Retail Sales dataset into the workflow environment and verify successful execution.

STEP 4 Verify the Data Types

Inspect all columns to ensure numeric fields (Quantity, Price, Revenue) are typed as Integer or Double, and categorical fields (Category, Product Name) are set as String.

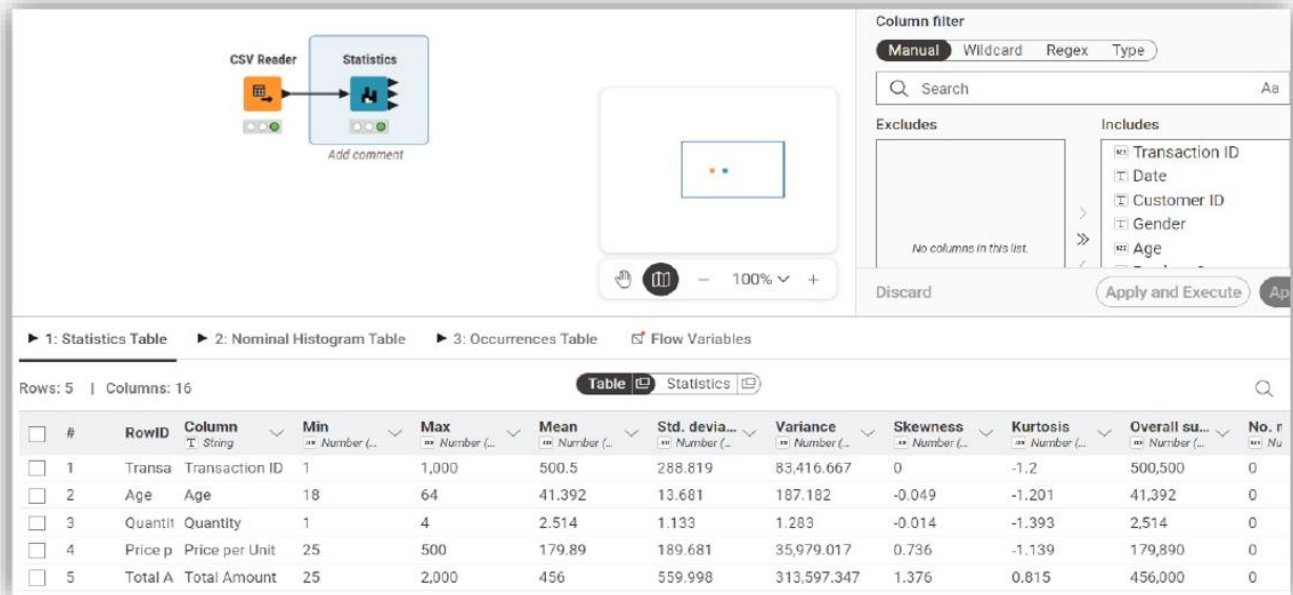


STEP 5 Generate Summary Statistics

Apply the Statistics node to compute descriptive measures — mean, minimum, maximum, sum, and standard deviation — for all numeric attributes.

STEP 6 Calculate Revenue Column

Use the Math Formula node to create a new Revenue column: **Revenue = Quantity × Price**. This derived column is used for all KPI calculations.



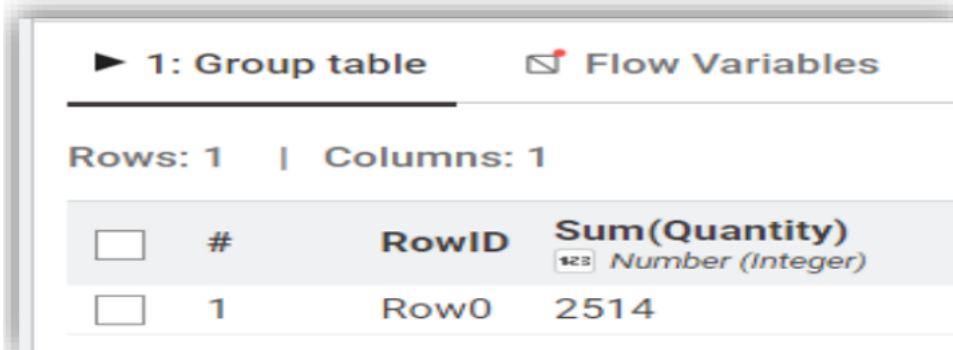
#	RowID	Column	Min	Max	Mean	Std. devia...	Variance	Skewness	Kurtosis	Overall su...	No. r
1	Transa	Transaction ID	1	1,000	500.5	288.819	83,416.667	0	-1.2	500,500	0
2	Age	Age	18	64	41.392	13.681	187.182	-0.049	-1.201	41,392	0
3	Quantit	Quantity	1	4	2.514	1.133	1.283	-0.014	-1.393	2,514	0
4	Price p	Price per Unit	25	500	179.89	189.681	35,979.017	0.736	-1.139	179,890	0
5	Total A	Total Amount	25	2,000	456	559.998	313,597.347	1.376	0.815	456,000	0

STEP 7 Total Revenue (KPI 1)

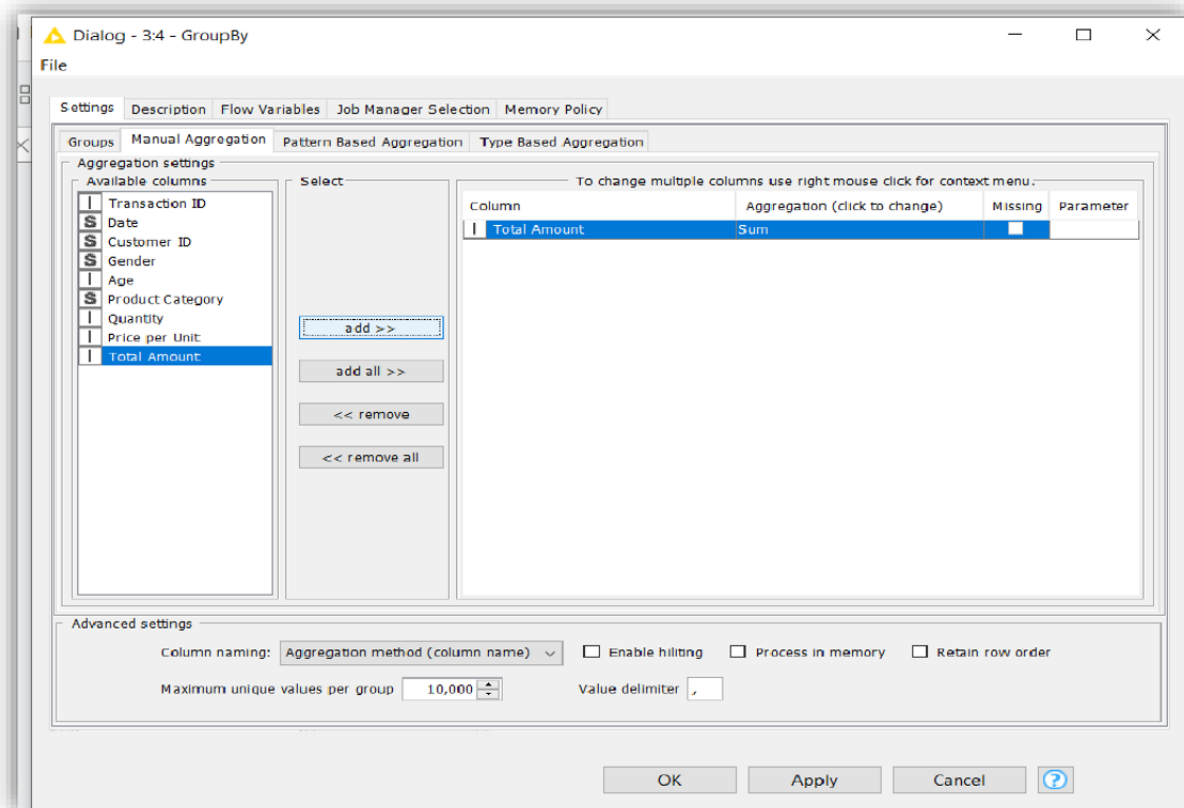
Add a GroupBy node with no grouping column and aggregate Revenue using Sum to compute overall sales revenue.

STEP 8 View Total Revenue Result

Execute the node and observe the single output value representing total revenue across all transactions.



#	RowID	Sum(Quantity)
1	Row0	2514



STEP 9 Revenue per Transaction

Add a GroupBy node grouped by Transaction ID and aggregate Revenue using Sum for each individual order.

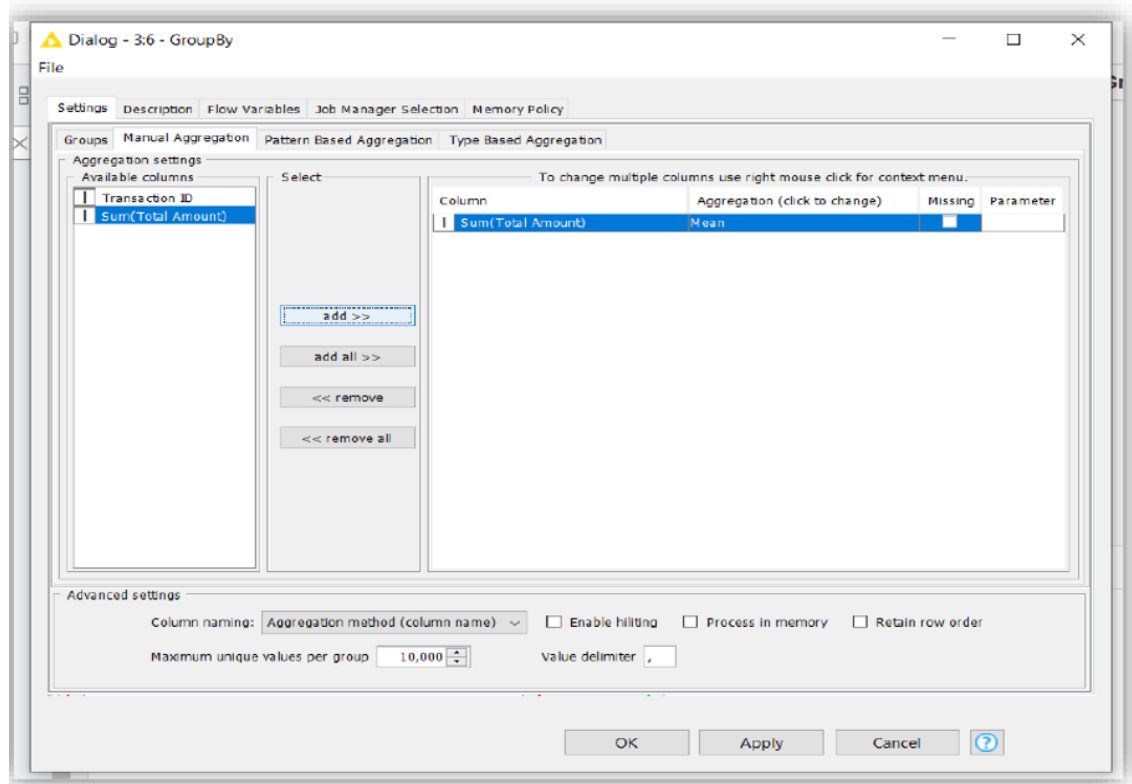
► 1: Group table Flow Variables

Rows: 1000 | Columns: 2

#	RowID	Transaction ID Number (Integer)	Sum(Total Amount) Number (Integer)
1	Row0	1	150
2	Row1	2	1000
3	Row2	3	30
4	Row3	4	500
5	Row4	5	100
6	Row5	6	30

**STEP
10****Average Order Value (KPI 2)**

Connect a second GroupBy node with no grouping column and calculate the Mean of aggregated revenue.

**STEP
11****View Average Order Value**

Execute and review the output showing average revenue per transaction.

► 1: Group table  Flow Variables

Rows: 1 | Columns: 1

☐	#	RowID	Mean(Sum(Total Amount)) .00 Number (Float)
☐	1	Row0	456

**STEP
12****Revenue by Category (KPI 3)**

Add a GroupBy node grouped by Product Category and aggregate Revenue using Sum for each category.

**STEP
13****Sort by Highest Revenue Category**

Connect a Sorter node configured for descending order to rank categories from highest to lowest.

Step 12: Calculate Category Revenue (KPI 3 – Revenue by Category)

Add a GroupBy node and group by Product Category. Aggregate Revenue using Sum to compute total earnings for each category.

The screenshot shows the Alteryx Designer interface for Step 12. The workflow consists of a CSV Reader node connected to a Statistics node, a Math Formula node, and three GroupBy nodes. The final output is a table titled "1: Group table" with 3 rows and 2 columns. The table displays the sum of total amounts for each product category.

#	RowID	Product Category	Sum(Total Amount)
1	Row0	Beauty	143515
2	Row1	Clothing	155580
3	Row2	Electronics	156905

Step 13: Sort to Identify Highest Revenue Category

Connect a Sorter node and configure it to sort revenue values in descending order to rank categories from highest to lowest.

The screenshot shows the Alteryx Designer interface for Step 13. The workflow includes a CSV Reader, Statistics, Math Formula, three GroupBy nodes, and a Sorter node. The Sorter node is configured to sort by Sum(Total Amount) in descending order. The final output is a table titled "1: Sorted Table" with 3 rows and 2 columns. The table displays the sum of total amounts for each product category, sorted in descending order.

#	RowID	Product Category	Sum(Total Amount)
1	Row0	Beauty	143515
2	Row1	Clothing	155580
3	Row2	Electronics	156905

Lab 2: Customer Churn Prediction

Real-World Scenario: An e-commerce business aims to identify customers likely to discontinue its services. Undetected churn leads to revenue loss and missed retention opportunities. This lab applies the CRISP-DM methodology with Orange Data Mining to build a predictive churn model.

Dataset: Churn_Modelling.csv

10,000 customer records with 14 attributes. The target column is **Exited** (1 = Churned, 0 = Retained). Input features include Credit Score, Geography, Gender, Age, Tenure, Account Balance, Number of Products, Credit Card Ownership, Active Membership Status, and Estimated Salary. Columns RowNumber, CustomerId, and Surname are excluded during preprocessing.

Software: Orange Data Mining

Orange is a free, open-source visual data mining tool for building machine learning workflows by connecting widgets on a canvas — supporting data import, preprocessing, classification, evaluation, and visualisation without coding.

Learning Objectives

- 1 Import and inspect a CSV dataset using File and Data Table widgets.
.
- 2 Remove unnecessary columns and assign the target variable using Select Columns.
.
- 3 Normalise numerical data and encode categorical features using Preprocess.
.
- 4 Train Logistic Regression and Random Forest classification models.
.
- 5 Compare model performance using Test and Score with 10-fold cross-validation.
.
- 6 Analyse results using Confusion Matrix and ROC Analysis.
.
- 7 Compute business KPIs: Churn Rate and Retention Rate.
.

Procedure

STEP 1 Load the Dataset

Use the File widget to load Churn_Modelling.csv and connect it to a Data Table widget to verify all 10,000 rows and 14 columns are loaded correctly.

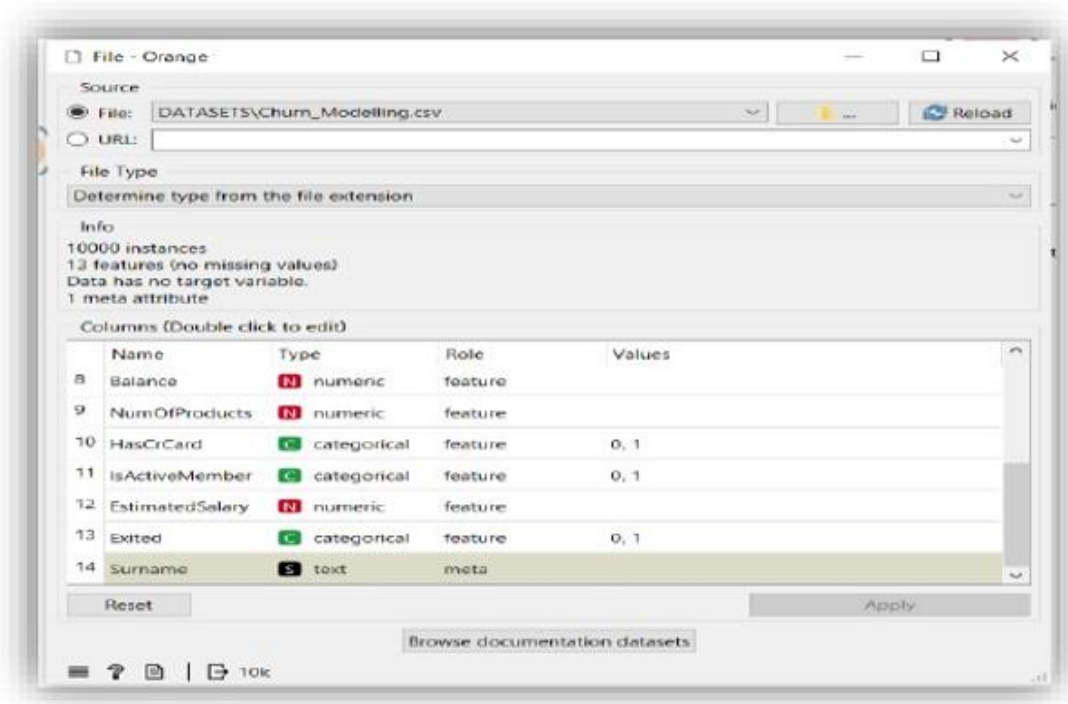
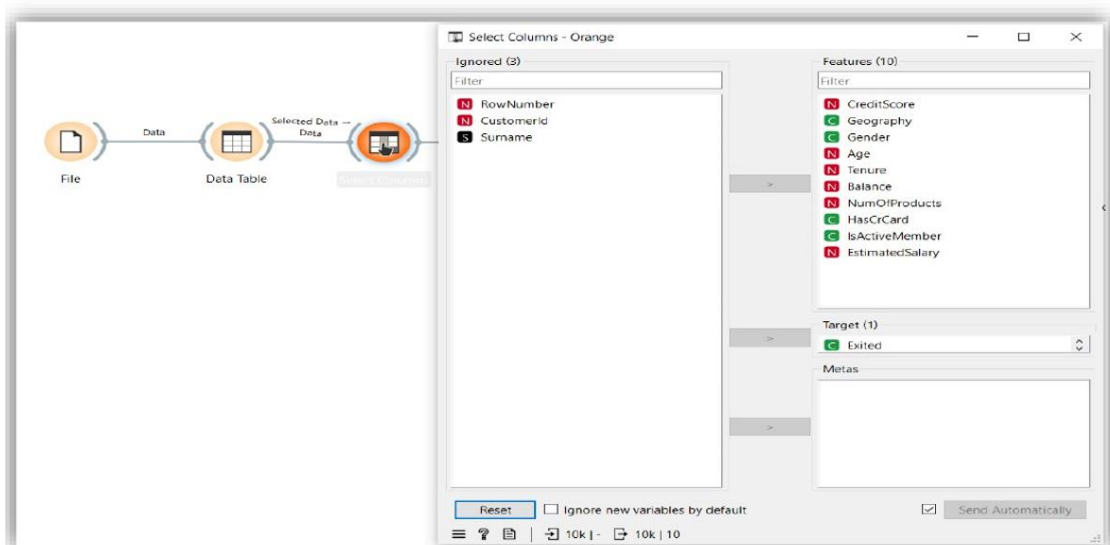


Figure 2.1 — Orange File widget loading Churn_Modelling.csv

STEP 2 Select Columns

Connect to a Select Columns widget. Move RowNumber, CustomerId, and Surname to the Ignore list. Set Exited as the Target class and retain all remaining features.



STEP 3 Preprocess

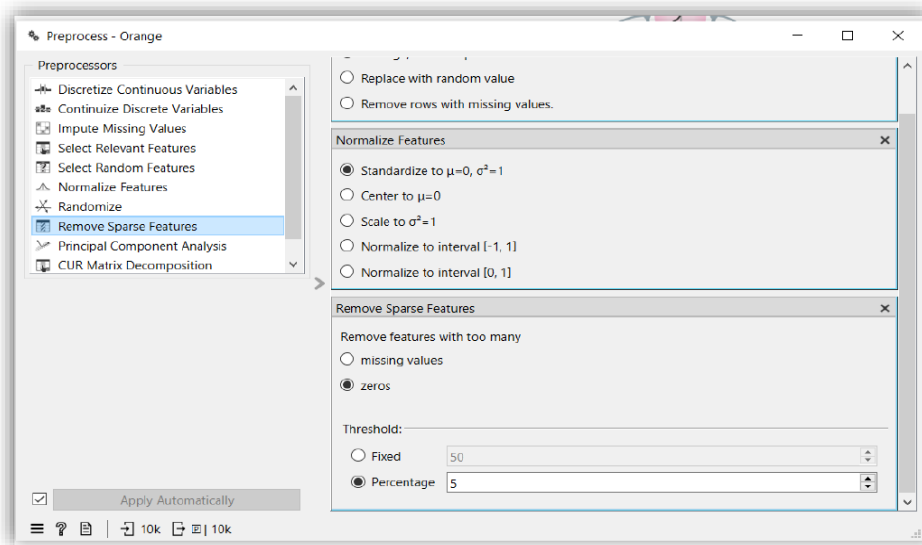
Connect to a Preprocess widget and enable Min-Max normalisation to scale all numeric features between 0 and 1.

STEP 4 Add Models

Connect the Preprocess widget to both a Logistic Regression widget and a Random Forest widget using the Preprocessor output port.

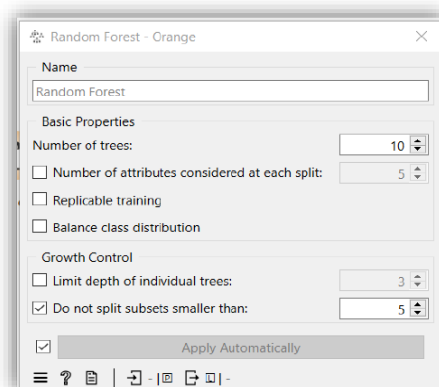
Step 3: Preprocess

Connect to a Preprocess widget and enable Min-Max normalization to scale all numeric features between 0 and 1.



Step 4: Add Models

Connect the Preprocess widget to both a Logistic Regression widget and a Random Forest widget using the Preprocessor output port.



STEP 5 Test and Score

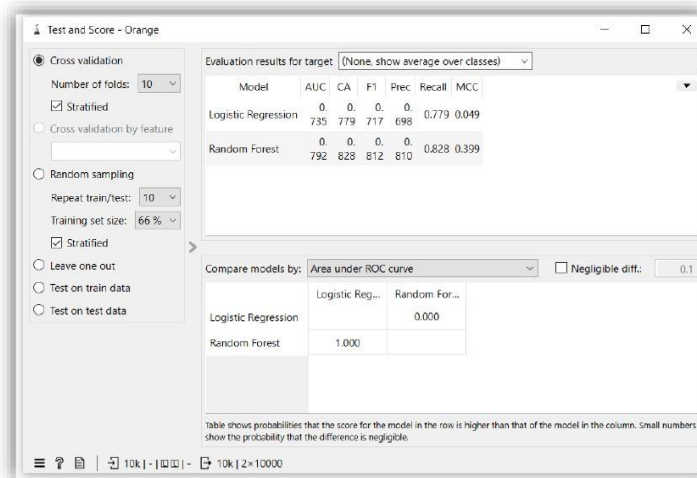
Connect both learners and the preprocessed data to the Test and Score widget. Set evaluation to Cross Validation with 10 folds and enable Stratified splitting.

STEP 6 Evaluate

Connect Test and Score to a ROC Analysis widget and Confusion Matrix widget to visualise model performance.

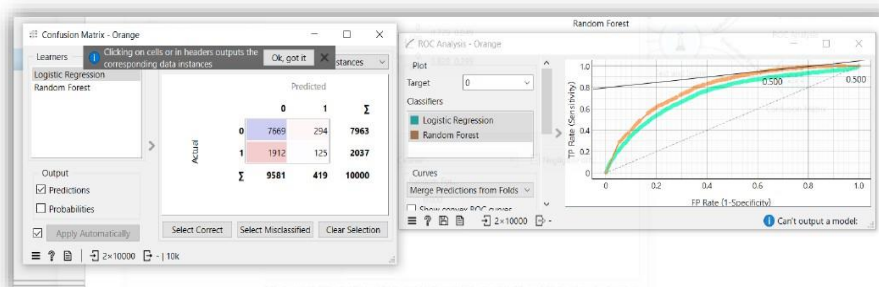
Step 5: Test and Score

Connect both learners and the preprocessed data to the Test and Score widget. Set evaluation to Cross Validation with 10 folds and enable Stratified splitting.



Step 6: Evaluate

Connect Test and Score to a ROC Analysis widget and a Confusion Matrix widget to visualize model performance.



Results & KPIs

KPI	Result
Churn Rate	20.37%
Retention Rate	79.63%
Logistic Regression AUC	0.735
Random Forest AUC	0.792
Random Forest Accuracy	82.8%
Random Forest F1 Score	0.812

Key Insight: Random Forest outperformed Logistic Regression across all metrics. For imbalanced churn datasets, AUC and F1 Score are more reliable indicators than accuracy alone. The model can be deployed to score customers and trigger early retention actions for high-risk segments.

Lab 3: Data Cleaning

6/16/2026

Real-World Scenario: Raw healthcare data often includes errors such as missing information, duplicate records, and unrealistic billing values. These issues reduce the accuracy of medical analysis. An organised data cleaning process is necessary to convert raw patient records into a reliable, analysis-ready form.

Dataset: Healthcare Records

55,500 rows and 15 columns. Attributes include Patient Name, Age, Gender, Blood Type, Medical Condition, Billing Amount, Admission Type, and Medication. Cleaning focus areas: text standardisation, missing value handling, numeric outlier removal, and duplicate elimination.

Software: Python — Pandas & NumPy

Learning Objectives

- 1

•

Detect and handle missing values in healthcare records.
- 2

•

Remove duplicate rows to improve data consistency.
- 3

•

Standardise text values and filter invalid numerical entries.
- 4

•

Prepare a clean dataset suitable for healthcare analytics.
- 5

•

Calculate KPIs: % Missing Values Reduced, Duplicate Removal %, Data Quality Score.

Procedure

STEP 1

Environment Setup & Data Loading

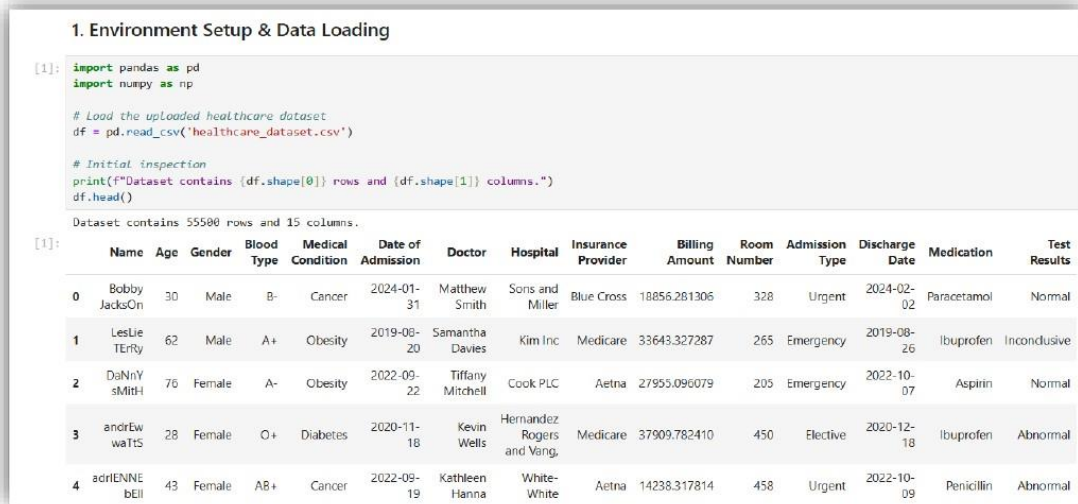
Import Pandas and NumPy, then load the healthcare dataset and perform an initial inspection of its structure.

- Data Quality Score

Procedure & Steps

Step 1: Environment Setup & Data Loading

Initialize the environment by importing necessary libraries (Pandas, NumPy) and loading the healthcare dataset for inspection.

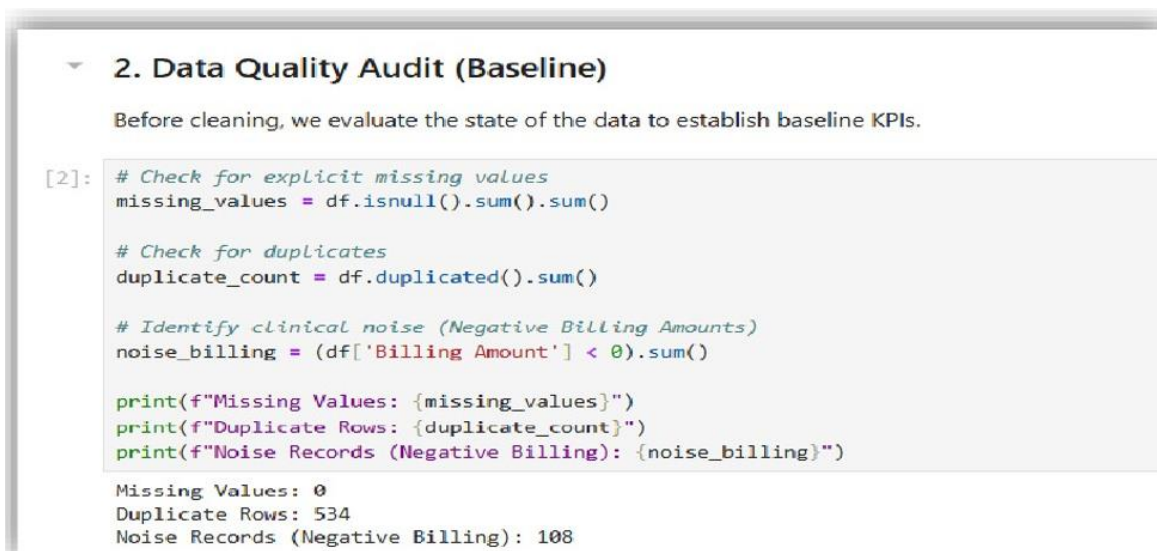


19

Figure 3.1 — Environment setup and dataset loaded (55,500 rows × 15 columns)

STEP 2 Initial Data Inspection (Quality Audit)

Check the dataset dimensions and identify initial issues: missing values across columns, duplicate row count, and noise records such as negative billing amounts.



STEP 3 Duplicate Record Removal

Scan the dataset for identical patient entries and remove redundant rows to ensure uniqueness. Result: 534 duplicates removed (0.96% reduction).

3. Data Cleaning Implementation

3.1. Removing Duplicates

Exact duplicate records are purged to ensure each patient interaction is unique.

```
[3]: initial_len = len(df)
df = df.drop_duplicates()
final_len = len(df)

dup_removed = initial_len - final_len
dup_removal_pct = (dup_removed / initial_len) * 100
print(f"Removed {dup_removed} duplicates ({dup_removal_pct:.2f}% reduction).")

Removed 534 duplicates (0.96% reduction).
```

STEP 4 Clinical Noise Filtering & Standardising

Standardise text casing (Title Case for names) and remove illogical numeric outliers such as billing amounts ≤ 0 .

3.2. Noise Filtering & Standardization

We address two types of noise:

1. **Text Noise:** Names like 'Bobby JacksOn' are standardized to Title Case.
2. **Numerical Noise:** Negative 'Billing Amount' values are corrected (absolute value) or filtered.

```
[4]: # Standardize Name Casing
df['Name'] = df['Name'].str.title()

# Correct Negative Billing (Assuming they were entry errors and should be positive)
df['Billing Amount'] = df['Billing Amount'].abs()

# Trim white spaces from categorical columns
categorical_cols = df.select_dtypes(include=['object']).columns
for col in categorical_cols:
    df[col] = df[col].str.strip()

print("Noise filtering complete. Names standardized and negative billing corrected.")
df.head()
```

Noise filtering complete. Names standardized and negative billing corrected.

```
[4]:
```

	Name	Age	Gender	Blood Type	Medical Condition	Date of Admission	Doctor	Hospital	Insurance Provider	Billing Amount	Room Number	Admission Type	Discharge Date	Medication	Test Results
0	Bobby Jackson	30	Male	B-	Cancer	2024-01-21	Matthew Smith	Sons and Miller	Blue Cross	18856.281306	328	Urgent	2024-02-02	Paracetamol	Normal
1	Leslie Terry	62	Male	A+	Obesity	2019-08-20	Samantha Davies	Kim Inc	Medicare	33643.327287	265	Emergency	2019-08-26	Ibuprofen	Inconclusive
2	Danny Smith	76	Female	A-	Obesity	2022-09-22	Tiffany Mitchell	Cook PLC	Aetna	27955.096079	205	Emergency	2022-10-07	Aspirin	Normal
3	Andrew Watts	28	Female	O+	Diabetes	2020-11-18	Kevin Wells	Hernandez Rogers and Vang	Medicare	37909.782410	450	Elective	2020-12-18	Ibuprofen	Abnormal
4	Adrienne Bell	43	Female	AB+	Cancer	2022-09-19	Kathleen Hanna	White-White	Aetna	14238.317814	458	Urgent	2022-10-09	Penicillin	Abnormal

STEP 5 KPI Calculation

Compute performance metrics to verify cleaning quality, including the final Data Quality Score.

```
final_missing = df.isnull().sum().sum()

# 2. Data Quality Score
# Calculated as: 1 - (Remaining Issues / Total Rows)
remaining_noise = (df['Billing Amount'] <= 0).sum()
quality_score = ((len(df) - remaining_noise) / len(df)) * 100

kpis = pd.DataFrame({
    "KPI": ["% Missing Reduced", "Duplicate Removal %", "Data Quality Score"],
    "Value": ["100%", f"{dup_removal_pct:.2f}%", f"{quality_score:.2f}%"]
})

kpis
```

```
[5]:
```

	KPI	Value
0	% Missing Reduced	100%
1	Duplicate Removal %	0.96%
2	Data Quality Score	100.00%

STEP 6**Export Cleaned Data**

Save the final processed dataset as **cleaned_healthcare_data.csv** for future analysis.

Step 6: Export Cleaned Data

Save the final processed dataset as a new file (e.g., **cleaned_healthcare_data.csv**) for future analysis.

5. Export Cleaned Data

```
[6]: df.to_csv('cleaned_healthcare_data.csv', index=False)
      print("Cleaned dataset saved as 'cleaned_healthcare_data.csv'.")
      Cleaned dataset saved as 'cleaned_healthcare_data.csv'.
```

Figure 3.6 — Export confirmation

Lab 4: Data Preprocessing

Real-World Scenario: A banking institution must evaluate loan applicant risk before granting approvals. Raw loan datasets contain missing values, unequal data scales, and continuous variables requiring transformation before reliable predictive models can be built.

Dataset: Loan Applications — 20,000 Records

Key attributes: Age, Annual Income, Monthly Income, Debt-to-Income Ratio, Credit Score, Loan Amount, Interest Rate, Loan Term, and Installment Amount. After handling missing values, **8,231 clean records** remained for processing.

Software: RapidMiner Studio

Learning Objectives

- 1 Import a CSV dataset using the Retrieve operator.
.
- 2 Handle incomplete records using Replace Missing Values.
.
- 3 Normalise numerical features through Z-Score Normalisation.
.
- 4 Apply Z-Score Peak Transformation to reduce outlier effects.
.
- 5 Convert continuous variables into categories using Discretisation.
.
- 6 Generate descriptive summaries using the Statistics operator.
.
- 7 Measure preprocessing success via Variance Reduction and Distribution Improvement KPIs.
.

Procedure

STEP 1 Load the Dataset

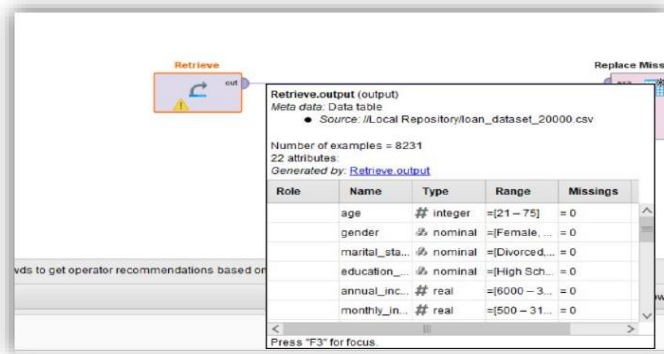
Use the Retrieve operator to load `loan_dataset_20000.csv` into the workflow and connect it to begin preprocessing.

- variance reduction
- Distribution Improvement

Procedure

Step 1: Load the Dataset

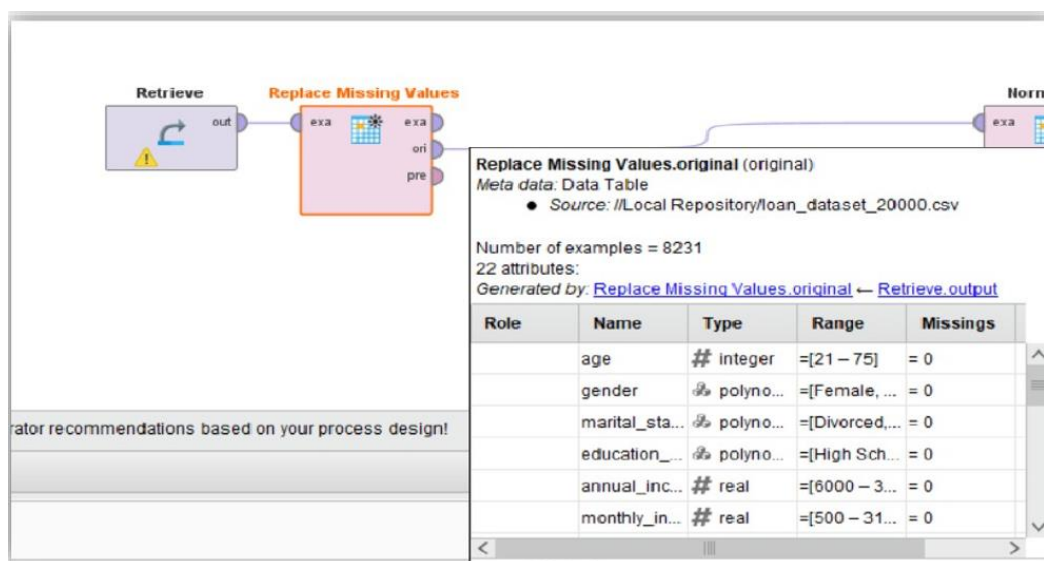
Use the Retrieve operator to load loan_dataset_20000.csv into the workflow. Connect it to the next operator to begin preprocessing.



24

STEP 2 Replace Missing Values

Connect to a Replace Missing Values operator, removing or imputing missing entries — reducing the dataset from 20,000 to 8,231 valid records.

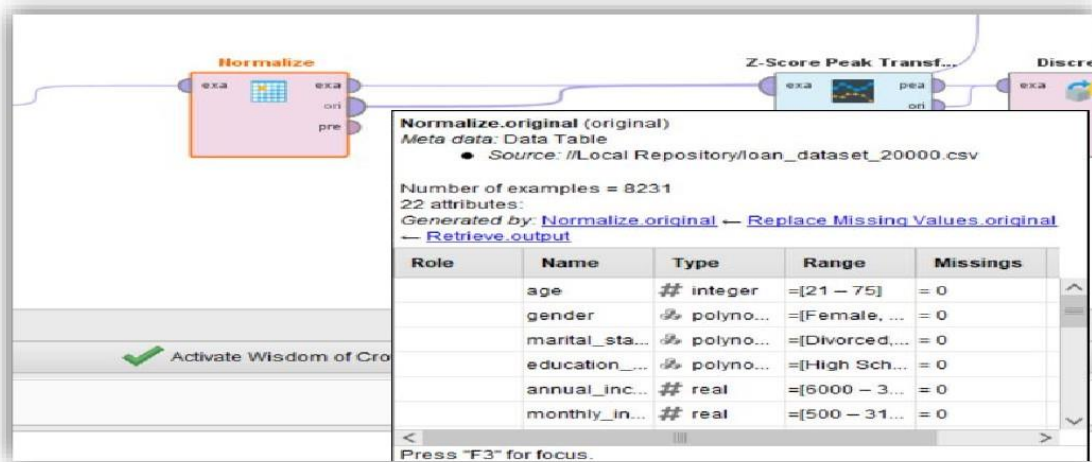


STEP 3 Normalise

Connect to a Normalise operator and apply Z-Score normalisation to scale all numeric features to mean=0 and standard deviation=1

Step 3: Normalize

Connect to a Normalize operator and apply Z-Score normalization. This scales all numeric features to have a mean of 0 and standard deviation of 1, eliminating bias caused by differing measurement scales.



25

Figure 4.3 — Z-Score Normalisation configured

STEP 4 Z-Score Peak Transformation

Identify and handle extreme outlier values that remain after normalisation, producing a cleaner distribution.

Result History | ExampleSet (Z-Score Peak Transformation) | ExampleSet (Normalize) | StatisticsObject (Statistics)

Open in Turbo Prep Auto Model Filter (8,231 / 8,231 examples): all

Row No.	age	annual_inc...	monthly_inc...	debt_to_inc...	credit_score	loan_amount	interest_rate	loan_term	installment	num_of_o...
1	0.068	-0.595	-0.595	0.546	1.438	-1.350	-1.178	1.488	-1.391	-1.337
2	-0.817	-1.085	-1.085	0.833	1.870	-0.032	-1.803	-0.672	0.043	0.870
3	0.952	-0.624	-0.624	0.795	-0.230	0.156	1.126	1.488	-0.222	-1.778
4	-1.259	0.409	0.409	-0.920	1.366	-0.290	0.123	1.488	-0.617	-0.895
5	-0.564	0.661	0.661	-0.987	0.647	-1.715	-1.007	-0.672	-1.611	0.429
6	-0.311	0.337	0.337	1.896	1.883	-0.058	-0.251	-0.672	0.114	-0.454
7	-1.069	-0.223	-0.223	-0.106	0.618	1.123	1.110	-0.672	1.490	-0.895
8	-0.374	0.515	0.515	0.316	-1.453	0.105	1.955	1.488	-0.198	-0.454
9	1.015	0.912	0.912	-0.738	-3.438	-0.741	0.769	-0.672	-0.557	-0.454
10	0.131	-0.108	-0.108	-1.054	1.064	0.490	-0.259	-0.672	0.684	0.429
11	0.636	-0.828	-0.828	-0.355	-0.893	0.340	0.574	-0.672	0.593	1.312
12	-1.638	-1.038	-1.038	-0.527	0.431	-0.643	0.509	-0.672	-0.463	0.429
13	-0.438	-0.911	-0.911	2.375	0.374	0.398	0.444	-0.672	0.645	-0.895
14	-1.006	-1.221	-1.221	0.431	0.187	-1.100	-0.068	-0.672	-0.966	1.753
15	1.015	0.166	0.166	0.172	-0.992	-1.157	-0.775	1.488	-1.254	-1.778
16	-0.185	-0.535	-0.535	-0.431	-1.812	-1.043	1.049	-0.672	-0.875	-0.895
17	0.889	-0.319	-0.319	-0.833	0.086	-0.529	-0.304	-0.672	-0.378	0.870
18	-0.753	0.141	0.141	0.392	1.827	0.928	-2.067	-0.672	0.962	-0.012

ExampleSet (8,231 examples, 0 special attributes, 22 regular attributes)

Step 4: Z-Score Peak Transformation

Connect to a Z-Score Peak Transformation operator. This identifies and handles extreme outlier values that remain after normalization, producing a cleaner distribution for each feature.

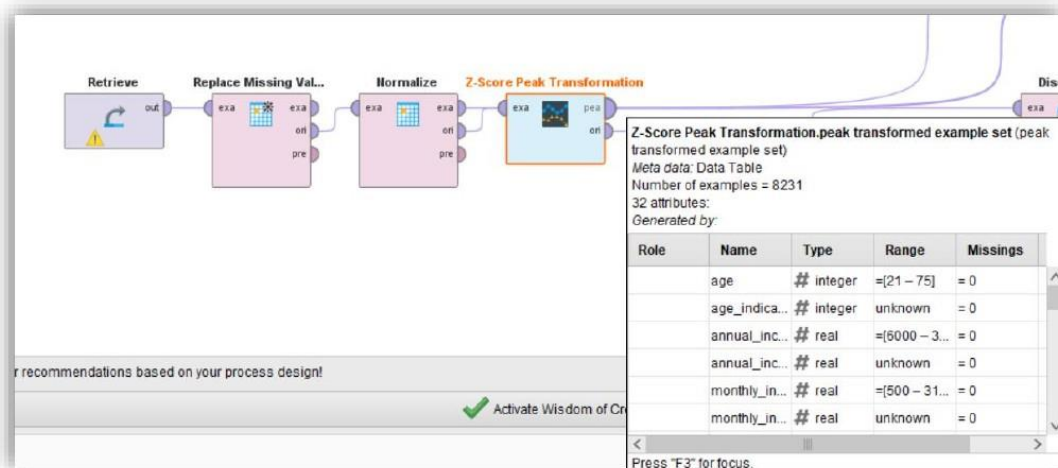


Figure 4.4 — Z-Score Peak Transformation output showing normalised feature values

STEP 5 Discretise

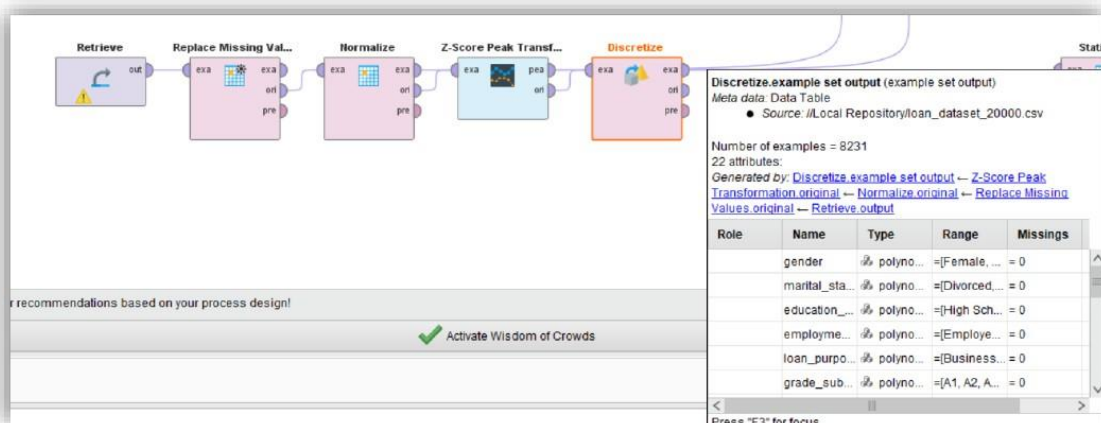
Connect to a Discretise operator to convert continuous numeric columns into categorical bins for rule-based or classification models.

Row No.	age	age_indicator	age_peaked	annual_inco...	annual_inco...	annual_inco...	monthly_inc...	monthly_inc...	monthly_inc...	debt_to_in
1	49	?	?	26181.800	?	?	2181.820	?	?	0.234
2	35	?	?	11873.840	?	?	989.490	?	?	0.284
3	63	?	?	25326.440	?	?	2110.540	?	?	0.260
4	28	?	?	55558.800	?	?	4629.980	?	?	0.081
5	39	?	?	62922.050	?	?	5243.500	?	?	0.074
6	43	?	?	53439.890	?	?	4453.320	?	?	0.375
7	31	?	?	37067.580	?	?	3088.960	?	?	0.166
8	42	?	?	58633.230	?	?	4886.100	?	?	0.210
9	64	?	?	70246.140	?	?	5853.840	?	?	0.100
10	50	?	?	40422.150	?	?	3368.510	?	?	0.067
11	58	0	?	19364.050	0	?	1613.670	0	?	0.140
12	22	0	?	13220.540	0	?	1101.710	0	?	0.122
13	41	0	?	16936.710	0	?	1411.390	0	?	0.425
14	32	0	?	7884.140	0	?	657.010	0	?	0.222
15	64	0	?	48453.690	0	?	4037.810	0	?	0.195
16	45	0	?	27944.460	0	?	2328.700	0	?	0.132
17	62	0	?	34248.840	0	?	2854.070	0	?	0.090
18	36	0	?	47690.490	0	?	3974.960	0	?	0.218

ExampleSet (8,231 examples, 0 special attributes, 48 regular attributes)

Step 5: Discretize

Connect to a Discretize operator. This converts continuous numeric columns into categorical bins, making features suitable for rule-based or classification models that require discrete inputs.



27

Figure 4.5 — Discretise operator output with binned attributes

STEP 6 Generate Statistics

Connect all outputs to a Statistics operator for a summary report: minimum, maximum, average, and standard deviation.

Step 6: Generate Statistics

Connect all result outputs to a Statistics operator. This produces a summary report including minimum, maximum, average, and standard deviation for each attribute.

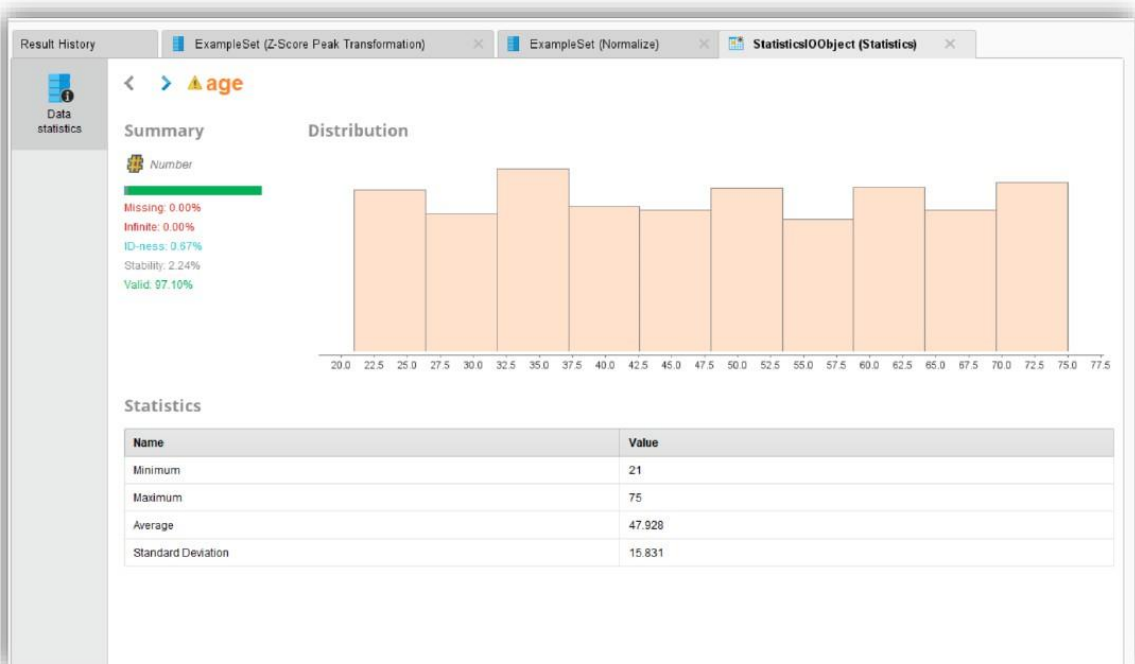
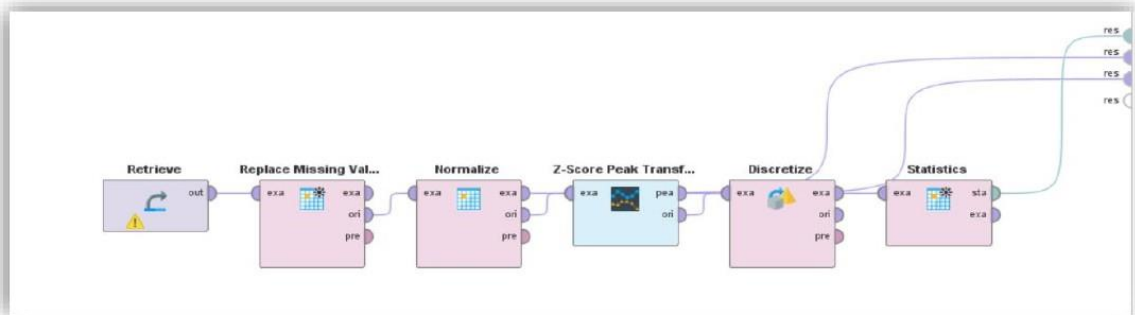


Figure 4.6 — Complete preprocessing pipeline and Statistics summary for Age

Results & KPIs

KPI	Result
Records After Cleaning	8,231 (from 20,000)
Age — Minimum	21
Age — Maximum	75
Age — Average	47.928
Age — Standard Deviation	15.831
Missing Values After Processing	0.00%
Valid Data Rate	97.10%

Lab 5: Market Basket Analysis

Real-World Scenario: Supermarkets process vast numbers of daily purchases. Without pattern analysis, businesses miss opportunities for strategic product placement, promotional bundles, and cross-selling. Discovering frequently co-purchased products helps retailers increase average basket value.

Dataset: Groceries Dataset

Member-based shopping transaction records. Attributes: Member_number, Date, ItemDescription. Each row represents one purchased item; records are grouped by member and date to form 14,963 unique transactions.

Software: Python — Pandas, itertools, collections

Learning Objectives

- 1 Identify frequent itemsets appearing regularly in customer baskets.
- 2 Perform association rule mining using Support, Confidence, and Lift.
- 3 Compare manual metric calculations with program-generated results.
- 4 Evaluate cross-selling potential using High Lift (>1) and strong Confidence values.

Procedure

STEP 1 Data Preparation & Transaction Grouping

Group raw CSV data by Member_number and Date to form unique transactions — converting individual item records into complete shopping baskets.

```
1. Data Preparation

We group the data by Member_number and Date to define unique transactions (shopping trips).

[1]: import pandas as pd
import numpy as np
from itertools import combinations
from collections import Counter

# Load dataset
df = pd.read_csv('Groceries_dataset.csv')

# Create transactions list
df['Transaction_ID'] = df['Member_number'].astype(str) + "_" + df['Date']
transactions = df.groupby('Transaction_ID')['ItemDescription'].apply(list).tolist()

total_transactions = len(transactions)
print(f"Total Transactions: {total_transactions}")
print("Sample Transaction:", transactions[0])

Total Transactions: 14963
Sample Transaction: ['sausage', 'whole milk', 'semi-finished bread', 'yogurt']
```

STEP 2 Manual Calculation of Support, Confidence & Lift

Manually calculate the three metrics for a specific item pair to validate the logic.

For the pair **whole milk** → **yogurt**: Support(A,B)

Step 2: Manual Calculation of Support, Confidence & Lift

Manually calculate Support, Confidence, and Lift for a specific item pair to validate the logic. Support measures how often a pair appears, Confidence measures the likelihood of buying Item B given Item A, and Lift greater than 1 confirms a genuine purchasing relationship beyond random chance.

2. Manual Calculation of Support, Confidence, and Lift

We will manually calculate the metrics for a specific pair (e.g., 'whole milk' and 'yogurt') to validate the logic.

```
[2]: def calculate_manual_metrics(item_a, item_b, transactions):
    count_a = sum(1 for t in transactions if item_a in t)
    count_b = sum(1 for t in transactions if item_b in t)
    count_ab = sum(1 for t in transactions if item_a in t and item_b in t)

    support_a = count_a / total_transactions
    support_b = count_b / total_transactions
    support_ab = count_ab / total_transactions

    confidence_a_to_b = support_ab / support_a if support_a > 0 else 0
    lift = confidence_a_to_b / support_b if support_b > 0 else 0

    return {
        "Support(A,B)": support_ab,
        "Confidence(A->B)": confidence_a_to_b,
        "Lift": lift
    }

metrics = calculate_manual_metrics('whole milk', 'yogurt', transactions)
print("Metrics for {whole milk -> yogurt}:")
for k, v in metrics.items():
    print(f"{k}: {v:.4f}")

Metrics for {whole milk -> yogurt}:
Support(A,B): 0.0112
Confidence(A->B): 0.0707
Lift: 0.8229
```

32

=0.0112,

Confidence(A→B)=0.0707, Lift=0.8229

STEP 3 Frequent Itemsets & Rule Generation

Compute metrics for every possible item pair across all transactions. Both directions ($A \rightarrow B$ and $B \rightarrow A$) are evaluated and ranked by Lift.

Step 3: Frequent Itemsets & Rule Generation

Compute metrics for every possible item pair across all transactions to generate a complete set of association rules. Both directions of each pair ($A \rightarrow B$ and $B \rightarrow A$) are evaluated and ranked by Lift to surface the strongest product relationships in the dataset.

```
3. Frequent Itemsets and Rule Generation

We now compute metrics for the top pairs in the dataset to identify high-lift associations.

[3]: # 1. Calculate Single Item Support
all_items = [item for sublist in transactions for item in sublist]
item_counts = Counter(all_items)
item_support = {item: count / total_transactions for item, count in item_counts.items()}

# 2. Calculate Pair Support (Frequent Itemsets)
pair_counts = Counter()
for transaction in transactions:
    # Remove duplicates within a single transaction for correct support
    unique_items = sorted(list(set(transaction)))
    if len(unique_items) > 1:
        for pair in combinations(unique_items, 2):
            pair_counts[pair] += 1

# 3. Generate Association Rules
rules = []
for pair, count in pair_counts.items():
    item_a, item_b = pair
    support_ab = count / total_transactions

    # Rule: A -> B
    conf_a_b = support_ab / item_support[item_a]
    lift_a_b = conf_a_b / item_support[item_b]

    # Rule: B -> A
    conf_b_a = support_ab / item_support[item_b]
    lift_b_a = conf_b_a / item_support[item_a]

    rules.append([item_a, item_b, support_ab, conf_a_b, lift_a_b])
    rules.append([item_b, item_a, support_ab, conf_b_a, lift_b_a])

rules_df = pd.DataFrame(rules, columns=['Antecedent', 'Consequent', 'Support', 'Confidence', 'Lift'])
rules_df = rules_df.sort_values(by='Lift', ascending=False).reset_index(drop=True)

print("Top 10 High Lift Rules:")
```

```
print("Top 10 High Lift Rules:")
rules_df.head(10)
```

	Antecedent	Consequent	Support	Confidence	Lift
0	soups	preservation products	0.000067	0.020833	311.729167
1	preservation products	soups	0.000067	1.000000	311.729167
2	artif. sweetener	baby cosmetics	0.000067	0.034483	171.988506
3	baby cosmetics	artif. sweetener	0.000067	0.333333	171.988506
4	kitchen utensil	pasta	0.000067	1.000000	123.661157
5	pasta	kitchen utensil	0.000067	0.008264	123.661157
6	sparkling wine	rubbing alcohol	0.000067	0.021739	65.056522
7	rubbing alcohol	sparkling wine	0.000067	0.200000	65.056522
8	prosecco	honey	0.000067	0.052632	60.578947
9	honey	prosecco	0.000067	0.076923	60.578947

STEP 4 Filtering Significant Rules

Filter to isolate rules with Lift > 1 and Confidence > 0.05 — the most actionable cross-selling opportunities.

STEP 5 Strategic Conclusion

Identify high-lift item pairs as prime candidates for shelf co-location, promotional bundling, and targeted cross-selling.

Step 4: Filtering Significant Rules

Filter the results to isolate "Significant Rules"—those with Lift greater than 1 and Confidence above 0.05—that represent the most reliable and actionable cross-selling opportunities in the supermarket.

4. Summary of Cross-Sell Potential

We filter rules with **Lift > 1** and **Confidence > 0.05** to identify actionable supermarket strategies.

```
[4]: # Filtering for significant rules
significant_rules = rules_df[(rules_df['Lift'] > 1) & (rules_df['Confidence'] > 0.05)]

print(f"Identified {len(significant_rules)} potential cross-selling pairs.")
significant_rules.head(15)
```

Identified 1005 potential cross-selling pairs.

	Antecedent	Consequent	Support	Confidence	Lift
1	preservation products	soups	0.000067	1.000000	311.729167
3	baby cosmetics	artif.sweetener	0.000067	0.333333	171.908506
4	kitchen utensil	pasta	0.000067	1.000000	123.661157
7	rubbing alcohol	sparkling wine	0.000067	0.200000	65.056522
8	prosecco	honey	0.000067	0.052632	60.578947
9	honey	prosecco	0.000067	0.076923	60.578947
10	whisky	rum	0.000067	0.125000	58.449219
12	cookware	spices	0.000134	0.117647	44.008824
14	honey	artif.sweetener	0.000067	0.076923	39.609655
17	toilet cleaner	roll products	0.000067	0.200000	36.495122
18	decalcifier	curd cheese	0.000067	0.111111	36.142512
20	decalcifier	soups	0.000067	0.111111	34.636574
23	cooking chocolate	light bulbs	0.000067	0.066667	34.397701
24	honey	flower (seeds)	0.000134	0.153846	32.885714
26	cocoa drinks	artif.sweetener	0.000067	0.062500	32.247845

Step 5: Strategic Conclusion

Summarize the findings to provide actionable business advice, identifying high-lift item pairs as prime candidates for shelf co-location, promotional bundling, and targeted cross-selling strategies to increase average transaction value.

5. Conclusion

Based on the analysis:

- **Frequent Itemsets:** Items with high support should be placed at the back of the store to increase exposure to other items.
- **High Lift Rules:** Pairs with high lift (e.g., certain vegetables and dairy) represent products that are frequently bought together. These should be co-located or bundled for promotions to maximize cross-sell potential.

Lab 6: Apriori Algorithm

Real-World Scenario: Retail businesses process high volumes of daily purchases. Hidden buying relationships between product categories often go unnoticed without proper analytical methods. The Apriori algorithm identifies frequently co-purchased categories, enabling targeted promotions, product bundles, and improved customer retention.

Dataset: Retail Transaction Dataset

Attributes: CustomerID, ProductCategory, Quantity. Records grouped by CustomerID form unique customer baskets. The transaction matrix covers 95,215 customers across 4 product categories: Books, Clothing, Electronics, and Home Decor.

Software: Python — mlxtend (apriori, association_rules)

Learning Objectives

- 1 Convert raw purchase data into a binary customer-item basket matrix.
- 2 Discover frequent itemsets appearing together in customer purchase histories.
- 3 Generate association rules and evaluate using Support, Confidence, and Lift.
- 4 Analyse how changes in minimum support threshold affect the number of rules generated.

Procedure

STEP 1 Environment Setup & Data Preparation

Install mlxtend and load the dataset. Group by CustomerID and ProductCategory, then encode quantities into binary values (1=purchased, 0=not) to build the transaction matrix.

```
...set_index("CustomerID"))

# Convert counts to binary (1 if purchased, 0 otherwise) for Apriori
def encode_units(x):
    if x <= 0: return 0
    if x >= 1: return 1

basket_sets = basket.applymap(encode_units)

print(f"Transaction Matrix Shape: {basket_sets.shape}")
basket_sets.head()
```

C:\Users\Qadri Laptop\AppData\Local\Temp\ipykernel_7800\3642429727.py:19: FutureWarning: DataFrame.applymap has been deprecated.

```
basket_sets = basket.applymap(encode_units)
Transaction Matrix Shape: (95215, 4)
```

```
[3]: ProductCategory Books Clothing Electronics Home Decor
      CustomerID
      14          1          0          0          0
      42          0          0          0          1
      49          0          0          1          0
      59          0          1          0          1
      65          0          1          0          0
```

STEP 2 Frequent Itemset Generation

Apply the Apriori algorithm with minimum support of 0.01 to identify categories appearing in at least 1% of customer baskets.

Step 2: Frequent Itemset Generation

Apply the Apriori algorithm with a minimum support of 0.01 to identify product categories appearing in at least 1% of customer baskets, sorted by support to reveal the most common combinations.

2. Frequent Itemset Generation

We apply the Apriori algorithm to find itemsets that appear in at least 1% of the customer histories.

```
[4]: # Generate frequent itemsets with a minimum support of 0.01 (1%)
frequent_itemsets = apriori(basket_sets, min_support=0.01, use_colnames=True)

# Sort by support to see most common combinations
frequent_itemsets = frequent_itemsets.sort_values(by='support', ascending=False)
frequent_itemsets.head(10)
```

C:\Users\Qadri\Laptop\anaconda3\envs\new\Lib\site-packages\mlxtend\frequent_patterns\fpcommon.py:175: DeprecationWarning: DeprecationWarning: result in worse computational performance and their support might be discontinued in the future. Please use a DataWarning.warn(

```
[4]:
```

	support	itemsets
0	0.260012	(Books)
1	0.259791	(Clothing)
2	0.259192	(Electronics)
3	0.258762	(Home Decor)

Step 3: Rule Generation & Association Discovery

Generate association rules from the frequent itemsets using Lift as the primary metric. Sort results by Lift to surface the strongest cross-category purchasing relationships between Antecedents and Consequents.

3. Rule Generation & Association Discovery

STEP 4 KPI Analysis — Support Threshold Impact

Test four support thresholds (0.005, 0.01, 0.02, 0.05) to observe how threshold selection controls the breadth of discovered associations.

STEP 5 Strategic Conclusion

Identify dominant product combinations, high-lift cross-sell pairs, and the effect of support thresholds on rule discovery.

Test four support thresholds (0.005, 0.01, 0.02, 0.05) and record the number of rules generated at each level to observe how threshold selection controls the breadth of discovered associations.

Test four support thresholds (0.005, 0.01, 0.02, 0.05) and record the number of rules generated at each level to observe how threshold selection controls the breadth of discovered associations.

Step 5: Strategic Conclusion

Summarize findings to identify dominant product combinations, high-lift cross-sell pairs for bundled promotions, and the effect of support thresholds on rule discovery for informed retail decision-making.

- 1. Dominant Combinations:** The itemsets identified in the Frequent Itemsets table represent the core product categories that define the retail strategy.
- 2. Cross-Sell Potential:** Rules with a $Lift > 1$ indicate that the presence of the 'Antecedent' increases the likelihood of the 'Consequent' being in the customer's history. These pairs should be targeted for loyalty rewards or bundled digital coupons.
- 3. Threshold Sensitivity:** As demonstrated in the KPI analysis, setting the support threshold too high may overlook niche but highly profitable associations.

Lab 7: FP-Growth

Real-World Scenario: As retail transaction data grows, Apriori becomes slow due to repeated database scanning and large candidate set generation. FP-Growth compresses transaction data into an FP-Tree structure, enabling faster and more scalable frequent pattern mining while maintaining identical accuracy.

Dataset: New Retail Dataset

Attributes: Transaction_ID, Product_Type, Total_Purchases. Records grouped by Transaction_ID form unique baskets. The transaction matrix contains **294,461 transactions** across 33 unique product types.

Software: Python — mlxtend (fpgrowth, apriori, association_rules)

Learning Objectives

- 1 Construct an FP-Tree from the transaction matrix for efficient pattern discovery.
- 2 Compare execution performance of FP-Growth vs Apriori at the same support threshold.
- 3 Generate association rules from FP-Growth results using Lift as the evaluation metric.
- 4 Analyse KPIs: Execution Time, Number of Rules Generated, Efficiency Improvement.

Procedure

STEP 1 Environment Setup

Install mlxtend and import pandas, time, apriori, fpgrowth, and association_rules.

```
1. Environment Setup
We install and import the necessary libraries for Association Rule Mining.

[1]: # Install mlxtend if not already present
!pip install mlxtend

import pandas as pd
import time
from mlxtend.frequent_patterns import apriori, fpgrowth, association_rules

Requirement already satisfied: mlxtend in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (0.24.0)
Requirement already satisfied: scipy>=1.16.3 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from mlxtend) (1.16.3)
Requirement already satisfied: numpy>=2.3.5 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from mlxtend) (2.4.2)
Requirement already satisfied: pandas>=2.3.3 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from mlxtend) (2.3.3)
Requirement already satisfied: scikit-learn>=1.8.0 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from mlxtend) (1.8.0)
Requirement already satisfied: matplotlib>=3.10.8 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from mlxtend) (3.10.8)
Requirement already satisfied: joblib>=1.5.2 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from mlxtend) (1.5.2)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (1.3.3)
Requirement already satisfied: cycler>=0.10 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (4.60.1)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (1.4.9)
Requirement already satisfied: packaging>=20.0 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (25.0)
Requirement already satisfied: pillow>=8 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (12.0.0)
Requirement already satisfied: pyparsing>=3 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (3.2.5)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from pandas>=2.3.3->mlxtend) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from pandas>=2.3.3->mlxtend) (2025.2)
Requirement already satisfied: six>=1.5 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from python-dateutil>=2.7->matplotlib>=3.10.8->mlxtend) (1.17.0)
Requirement already satisfied: threadpoolctl>=3.2.0 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from scikit-learn>=1.8.0->mlxtend) (3.6.0)
```

STEP 2 Data Preprocessing

Load dataset, drop missing values in Transaction_ID and Product_Type, pivot into a binary transaction matrix of 294,461 transactions × 33 items.

Step 2: Data Preprocessing

Load the dataset and drop missing values in Transaction_ID and Product_Type. Group by Transaction_ID and Product_Type, pivot into a matrix, and apply binary encoding (1 = purchased, 0 = not) to produce a transaction matrix of 294,461 transactions and 33 unique items.

```
2. Data Preprocessing
We load the large retail dataset and transform it into a one-hot encoded 'basket' format. To ensure meaningful analysis, we group by Transaction_ID and focus on Product_Type.

[2]: # Load dataset
df = pd.read_csv('new_retail_data.csv')

# Clean missing values in critical columns
df = df.dropna(subset=['Transaction_ID', 'Product_Type'])

# Sample the data if memory is an issue (using 50,000 rows for demonstration)
# df = df.sample(50000, random_state=42)

# Pivot data to create a transaction matrix
basket = (df.groupby(['Transaction_ID', 'Product_Type'])['Total_Purchases']
          .sum().unstack().reset_index().fillna(0)
          .set_index('Transaction_ID'))

# Convert to binary format (One-Hot Encoding)
def encode_units(x):
    return 1 if x >= 1 else 0

basket_sets = basket.applymap(encode_units)

print(f"Transaction matrix ready: {basket_sets.shape[0]} transactions, {basket_sets.shape[1]} unique items.")

C:\Users\Qadri Laptop\AppData\Local\Temp\ipykernel_25136\2972179471.py:19: FutureWarning: DataFrame.applymap has been deprecated. Use DataFrame.map instead.
basket_sets = basket.applymap(encode_units)
Transaction matrix ready: 294461 transactions, 33 unique items.
```

Figure 7.2 — Transaction matrix ready: 294,461 transactions, 33 unique items

STEP 3 Apriori Execution

Run Apriori with minimum support of 0.005 and record execution time as a performance baseline.

```
3. Comparison Analysis: Execution Time & Rule Count
3.1. Apriori Execution

[3]: start_time = time.time()

# Min support set to 0.005 (0.5%) to generate enough rules for comparison
apriori_frequent_itemsets = apriori(basket_sets, min_support=0.005, use_colnames=True)
apriori_time = time.time() - start_time

print(f"Apriori Execution Time: {apriori_time:.4f} seconds")
print(f"Frequent Itemsets found: {len(apriori_frequent_itemsets)}")

C:\Users\Qadri Laptop\anaconda3\envs\new\lib\site-packages\mlxtend\frequent_patterns\fpcommon.py:175: DeprecationWarning: DataFrames with non
result in worse computational performance and their support might be discontinued in the future. Please use a DataFrame with bool type
warnings.warn(
Apriori Execution Time: 3.1329 seconds
Frequent Itemsets found: 33
```

STEP 4 FP-Growth Execution

Run FP-Growth on the same transaction matrix with minimum support of 0.005. FP-Growth builds a compressed FP-Tree and scans the database only twice.

STEP 5 Rule Generation & KPI Summary

Generate association rules from FP-Growth itemsets using Lift as the metric. Build a comparison table.

STEP 6 Final Insights

Both algorithms produce identical rule counts at the same support threshold, confirming FP-Growth's speed advantage does not compromise accuracy.

Step 4: FP-Growth Execution

Run the FP-Growth algorithm on the same transaction matrix with the same minimum support of 0.005. Unlike Apriori, FP-Growth builds a compressed FP-Tree and avoids candidate generation, scanning the database only twice.

```
3.2. FP-Growth Execution
Unlike Apriori, FP-Growth does not require candidate generation, instead using a compressed FP-Tree structure.

[4]: start_time = time.time()

fpgrowth_frequent_itemsets = fpgrowth(basket_sets, min_support=0.005, use_colnames=True)
fpgrowth_time = time.time() - start_time

print(f"FP-Growth Execution Time: {fpgrowth_time:.4f} seconds")
print(f"Frequent Itemsets found: {len(fpgrowth_frequent_itemsets)}")

C:\Users\Qadri-Laptop\anaconda3\envs\new\Lib\site-packages\mlxtend\frequent_patterns\fpcommon.py:175: DeprecationWarning: DataFrames with
result in worse computational performance and their support might be discontinued in the future. Please use a DataFrame with bool type
warnings.warn(
FP-Growth Execution Time: 5.3207 seconds
Frequent Itemsets found: 33
```

Step 5: Rule Generation & KPI Summary

Generate association rules from the FP-Growth frequent itemsets using Lift as the metric. Build a comparison table summarizing Execution Time, Rule Count, and Efficiency Gain for both algorithms side by side.

```
4. Rule Generation & KPI Summary
We generate association rules from the FP-Growth results and summarize the findings.

[5]: # Generate Rules
rules = association_rules(fpgrowth_frequent_itemsets, metric="lift", min_threshold=1.0)

# Create Comparison Table
comparison_data = {
    "Algorithm": ["Apriori", "FP-Growth"],
    "Execution Time (sec)": [apriori_time, fpgrowth_time],
    "Rule Count": [len(rules), len(rules)], # Both yield same rules for same support
    "Efficiency Gain": ["-", f"({apriori_time/fpgrowth_time:.2f})x Faster"]
}

summary_df = pd.DataFrame(comparison_data)
summary_df

[5]:
```

	Algorithm	Execution Time (sec)	Rule Count	Efficiency Gain
0	Apriori	3.132922	0	-
1	FP-Growth	5.320682	0	0.59x Faster

Step 6: Final Insights

Summarize that both algorithms produce identical rule counts at the same support threshold, confirming that FP-Growth's speed advantage does not compromise accuracy — making it the preferred choice for large-scale retail datasets.

Lab 8: Python Basics

Real-World Scenario: Data analysts often repeat the same preprocessing steps — loading, cleaning, encoding, and normalising — for different datasets. Without a structured solution, this creates repetitive code and inconsistent outputs. A reusable Python class organises all preprocessing tasks in one place, making data pipelines faster and easier to manage.

Dataset: Student Performance Dataset — 25,000 Records

Attributes: student_id, gender, school_type, attendance_percentage, study_hours, math_score, science_score, english_score, overall_score, final_grade. Each row represents the complete academic profile of one student.

Design Pattern: Object-Oriented Python (StudentDataProcessor class)

Learning Objectives

- 1 Design a reusable Python class for loading, cleaning, and preprocessing data.
- 2 Apply label encoding to categorical columns.
- 3 Use Min-Max Normalisation for score-related numerical columns.
- 4 Store metadata and summary results using lists and dictionary structures.
- 5 Evaluate KPIs: Code Efficiency, Reusability, and Better Workflow Management.

Procedure

STEP 1 Module Definition

Define the StudentDataProcessor class with an `__init__` method accepting a file path and initialising the DataFrame and a statistics dictionary. This encapsulates all processing logic within one reusable object.

```

A reusable class for processing student performance data.
"""
def __init__(self, file_path):
    self.file_path = file_path
    self.df = None
    self.stats_summary = {}

def load_data(self):
    """Loads the dataset into a pandas DataFrame."""
    self.df = pd.read_csv(self.file_path)
    print(f"Successfully loaded {len(self.df)} records.")
    return self.df

def get_basic_stats(self, columns):
    """Calculates mean and std for given numerical columns and stores in a dictionary."""
    for col in columns:
        self.stats_summary[col] = {
            'mean': self.df[col].mean(),
            'std': self.df[col].std()
        }
    return self.stats_summary

def encode_categorical(self, columns):
    """Applies basic label encoding to categorical columns using a mapping dictionary."""
    for col in columns:
        unique_vals = self.df[col].unique()
        mapping = {val: i for i, val in enumerate(unique_vals)}
        self.df[f'{col}_encoded'] = self.df[col].map(mapping)
        print(f"Encoded columns: {columns}")

def normalize_scores(self, score_cols):
    """Normalizes scores to a 0-1 range."""
    for col in score_cols:
        self.df[f'{col}_norm'] = (self.df[col] - self.df[col].min()) / (self.df[col].max() - self.df[col].min())

```

Step 2: Data Loading & Basic Statistics

Instantiate the class with `Student_Performance.csv` and call `load_data()` to read 25,000 records. Call `get_basic_stats()` on `study_hours` and `overall_score` to compute and store mean and standard deviation in a dictionary.

47

Figure 8.1 — *StudentDataProcessor* class definition with *load*, *stats*, *encode*, and *normalise* methods

STEP 2 Data Loading & Basic Statistics

Instantiate the class with `Student_Performance.csv`, call `load_data()` to read 25,000 records, then `get_basic_stats()` to compute mean and std for `study_hours` and `overall_score`.

STEP 3 Categorical Encoding

Call `encode_categorical()` on `gender`, `school_type`, and `final_grade`. Each unique value is mapped to a numeric integer, creating new `_encoded` columns.

STEP 4 Score Normalisation

Call `normalize_scores()` on `math_score`, `science_score`, and `english_score` applying Min-Max normalisation (0–1 range), creating new `_norm` columns.

2. Implementation & Testing

Now we instantiate our class and process the `Student_Performance.csv` file.

```
[2]: # Initialize processor
processor = StudentDataProcessor('Student_Performance.csv')

# 1. Load Data
df = processor.load_data()

# 2. Calculate Stats for study hours and overall score
stats = processor.get_basic_stats(['study_hours', 'overall_score'])
print("\nSummary Statistics:")
for col, val in stats.items():
    print(f"{col.title():<20}<td>Mean={val['mean']:.2f}</td> Std={val['std']:.2f}</td></tr></table>")

```

Step 3: Categorical Encoding

Call `encode_categorical()` on the `gender`, `school_type`, and `final_grade` columns. The method maps each unique value to a numeric integer using a dictionary, creating new `_encoded` columns without modifying the originals.

```
# 3. Encode Categorical variables for modeling
processor.encode_categorical(['gender', 'school_type', 'final_grade'])

```

Step 4: Score Normalization

Call `normalize_scores()` on `math_score`, `science_score`, and `english_score`. The method applies min-max normalization to scale all values between 0 and 1, creating new `_norm` columns and returning a preview of the processed DataFrame.

```
# 4. Normalize score columns
processor.normalize_scores(['math_score', 'science_score', 'english_score'])

print("\nProcessed Data Preview:")
processor.df[['student_id', 'gender_encoded', 'final_grade_encoded', 'math_score_norm']].head()

Successfully loaded 25000 records.

Summary Statistics:
Study_Hours: Mean=4.25, Std=2.17
Overall_Score: Mean=64.01, Std=18.93
Encoded columns: ['gender', 'school_type', 'final_grade']

Processed Data Preview:
[2]:
```

	student_id	gender_encoded	final_grade_encoded	math_score_norm
0	1	0	0	0.427
1	2	1	1	0.576
2	3	1	2	0.848
3	4	2	0	0.444
4	5	1	3	0.089

Figure 8.2 — Output: Study Hours Mean=4.25, Overall Score Mean=64.01; encoded and normalised columns preview

STEP 5 Reusability Showcase

Filter the DataFrame for students with attendance > 90% and compute their average overall score — without modifying any class code. Result: 4,957 high-attendance students, average score = 71.22.

STEP 6 Conclusion & KPIs

Summarise module performance across Code Efficiency, Reusability, and Object-Oriented Structure.

Step 5: Reusability Showcase

Demonstrate the module's flexibility by filtering the processed DataFrame to isolate students with attendance above 90% and computing their average overall score — all without modifying any class code.

3. Reusability Showcase

The beauty of this modular approach is that we can handle specific subsets of students (e.g., high achievers) using the same logic.

```
[3]: # Filter for students with high attendance
high_attendance_students = processor.df[processor.df['attendance_percentage'] > 90].copy()

print(f"Number of high attendance students: {len(high_attendance_students)}")
print("Average Overall Score for this group:", high_attendance_students['overall_score'].mean())

Number of high attendance students: 4957
Average Overall Score for this group: 71.22158563647368
```

Step 6: Conclusion & KPIs

Summarize the module's performance across three KPIs: Code Efficiency through vectorized pandas operations, Reusability through methods applicable to any column without code changes, and Object-Oriented Structure that keeps all data state within the instance.

4. Conclusion & KPIs

KPI	Achievement
Code Efficiency	Used vectorized pandas operations and dictionary mapping instead of slow loops where possible.
Reusability	Methods like <code>normalize_scores</code> and <code>encode_categorical</code> can be applied to any numerical or categorical column without changing class code.
Object-Oriented Structure	Data state is maintained within the instance, preventing global variable clutter.

Lab 9: Decision Tree (Customer Purchase Prediction)

Introduction

A **Decision Tree** is a supervised machine learning algorithm used for both classification and regression. It works like a flowchart: each internal node represents a decision based on a feature, each branch represents the outcome, and each leaf node represents the final prediction. Decision Trees are easy to understand and visualise because they mimic human decision-making.

Key Concepts

Entropy measures the impurity or randomness in data. Entropy = 0 indicates pure data; higher entropy means more mixed data.

Information Gain = Entropy(Parent) – Weighted Entropy(Children). The feature with the highest Information Gain is selected for splitting.

Gini Index = $1 - \sum(p_i^2)$. A lower Gini value indicates better purity.

Components

- **Root Node** — The topmost node representing the entire dataset.
- **Decision Node** — A node where data is split based on a condition.
- **Branch** — Represents the outcome of a decision.
- **Leaf Node** — The final node containing the class prediction.

Advantages & Limitations

Advantages	Limitations
• Easy to interpret • No preprocessing needed • Handles numeric & categorical • Works for classification & regression	• Can overfit training data • Sensitive to small data changes • Lower accuracy vs ensemble methods

Implementation

```
print("Libraries imported successfully!")  
  
... Libraries imported successfully!
```

Dataset

```
[15] # Features: Study Hours and Attendance Score
✓ Os X_dt = np.array([
    [2, 70], # Student 1: 2 hours study, 70% attendance -> Pass
    [3, 80], # Student 2: 3 hours study, 80% attendance -> Pass
    [1, 50], # Student 3: 1 hour study, 50% attendance -> Fail
    [4, 90], # Student 4: 4 hours study, 90% attendance -> Pass
    [1, 60], # Student 5: 1 hour study, 60% attendance -> Fail
    [3, 75], # Student 6: 3 hours study, 75% attendance -> Pass
    [0.5, 40], # Student 7: 0.5 hours study, 40% attendance -> Fail
    [4.5, 95], # Student 8: 4.5 hours study, 95% attendance -> Pass
    [1.5, 55], # Student 9: 1.5 hours study, 55% attendance -> Fail
    [2.5, 85], # Student 10: 2.5 hours study, 85% attendance -> Pass
])

# Target variable: Pass (1) or Fail (0)
y_dt = np.array([
    1, # Pass
    1, # Pass
    0, # Fail
    1, # Pass
    0, # Fail
    1, # Pass
    0, # Fail
    1, # Pass
    0, # Fail
    1 # Pass
])

print("Dataset created for Decision Tree:")
print("Features (X_dt - Study Hours, Attendance Score):\n", X_dt)
```

```
... Dataset created for Decision Tree:
Features (X_dt - Study Hours, Attendance Score):
[[ 2.  70.]
 [ 3.  80.]
 [ 1.  50.]
 [ 4.  90.]
 [ 1.  60.]
 [ 3.  75.]
 [ 0.5 40.]
 [ 4.5 95.]
 [ 1.5 55.]
 [ 2.5 85.]]
Target (y_dt - Pass/Fail):
[1 1 0 1 0 1 0 1 0 1]
```

Training

```
[16] X_train_dt, X_test_dt, y_train_dt, y_test_dt = train_test_split(X_dt, y_dt, test_size=0.3, random_state=42)
✓ Os

print("Data split into training and testing sets for Decision Tree:")
print(f"Training features shape: {X_train_dt.shape}")
print(f"Testing features shape: {X_test_dt.shape}")
print(f"Training target shape: {y_train_dt.shape}")
print(f"Testing target shape: {y_test_dt.shape}")

... Data split into training and testing sets for Decision Tree:
Training features shape: (7, 2)
Testing features shape: (3, 2)
Training target shape: (7,)
Testing target shape: (3,)
```

Testing on Data

```
[17] # Create a Decision Tree Classifier
✓ Os # random_state ensures reproducibility
dt_model = DecisionTreeClassifier(random_state=42)

# Train the model using the training data
dt_model.fit(X_train_dt, y_train_dt)

print("Decision Tree Classifier trained successfully!")

... Decision Tree Classifier trained successfully!
```

Prediction

```
# Make predictions on the test set
y_pred_dt = dt_model.predict(X_test_dt)

print("Predictions made on the test data.")

Predictions made on the test data.
```

Evaluation

```
[10] ✓ 0s print("\n--- Decision Tree Model Evaluation ---")

# 1. Print Predictions vs. Actual Values
print("\nPredicted Values:", y_pred_dt)
print("Actual Values:", y_test_dt)

# 2. Accuracy Score
accuracy_dt = metrics.accuracy_score(y_test_dt, y_pred_dt)
print(f"\nAccuracy Score: {accuracy_dt:.2f}")

# 3. Confusion Matrix
# This shows the counts of correct and incorrect predictions for each class.
# [[True Negatives, False Positives]
#  [False Negatives, True Positives]]
conf_matrix_dt = metrics.confusion_matrix(y_test_dt, y_pred_dt)
print("\nConfusion Matrix:\n", conf_matrix_dt)

# 4. Classification Report
# Provides precision, recall, and F1-score for each class (0=Fail, 1=Pass).
class_report_dt = metrics.classification_report(y_test_dt, y_pred_dt)
print("\nClassification Report:\n", class_report_dt)
```

```
...
--- Decision Tree Model Evaluation ---

Predicted Values: [0 1 1]
Actual Values: [0 1 1]

Accuracy Score: 1.00

Confusion Matrix:
[[1 0]
 [0 2]]

Classification Report:
precision    recall  f1-score   support

     0       1.00      1.00      1.00         1
     1       1.00      1.00      1.00         2

 accuracy          1.00
 macro avg          1.00
weighted avg          1.00
```

Graph

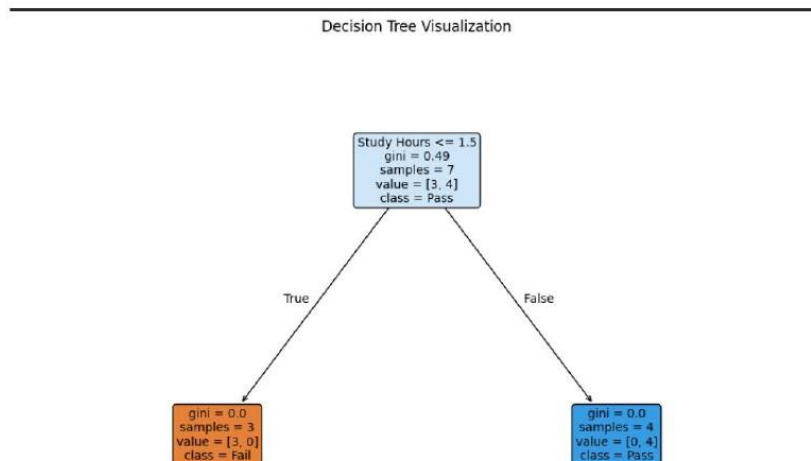


Figure 9.3 — Model evaluation (Accuracy: 1.00) and Decision Tree visualisation

Lab 10: Naïve Bayes & KNN (Email Spam Detection)

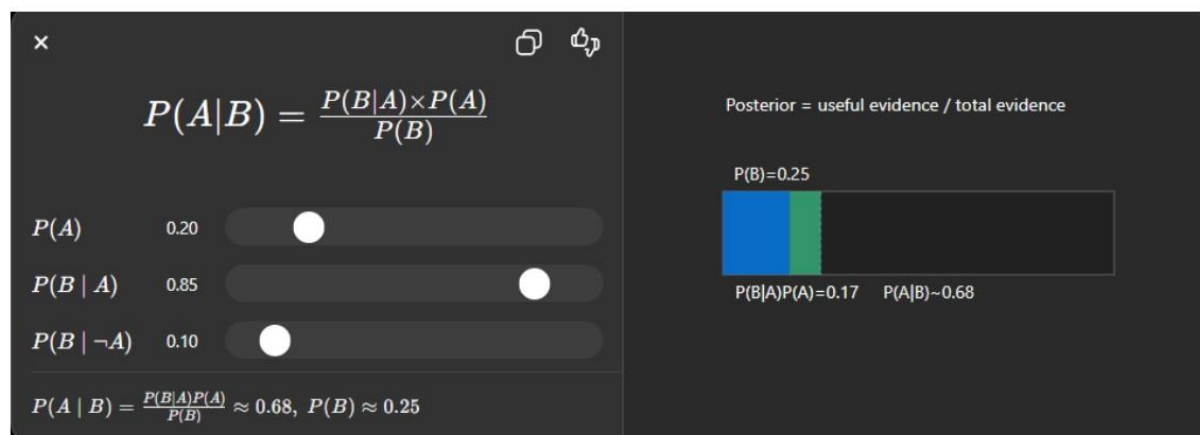
Naïve Bayes Algorithm

Naïve Bayes is a supervised machine learning algorithm based on Bayes' Theorem, primarily used for classification tasks such as spam detection, sentiment analysis, medical diagnosis, and document categorisation. It is called *Naïve* because it assumes all features are independent of each other.

tasks such as spam detection, sentiment analysis, medical diagnosis, and document categorization.

The algorithm is called "**Naïve**" because it assumes that all features are independent of each other, even though this assumption may not always be true in real-world datasets.

Bayes' Theorem



Where:

$P(A|B)$ = Probability of class A given evidence B

$P(B|A)$ = Probability of evidence B given class A

$P(A)$ = Prior probability of class A

$P(B)$ = Probability of evidence B

Working of Naïve Bayes

Calculate the prior probability of each class

Figure 10.1 — Bayes' Theorem: $P(A|B) = P(B|A) \times P(A) / P(B)$

Types of Naïve Bayes

- **Gaussian Naïve Bayes** — For continuous numerical data assuming normal distribution.
- **Multinomial Naïve Bayes** — For text classification using word frequencies.
- **Bernoulli Naïve Bayes** — For binary (0 or 1) features.

Example: Spam Detection

An email containing words like "**Win**", "**Prize**", "**Free**", "**Offer**" is scored against the Spam and Not Spam classes. The email is assigned to the class with the higher posterior probability.

Implementation — Naïve Bayes

```
print("Libraries imported successfully!")
```

```
... Libraries imported successfully!
```

Sample Dataset

```
# Features: Study Hours and Absences
X = np.array([
    [2, 1], # Student 1: 2 hours study, 1 absence
    [3, 0], # Student 2: 3 hours study, 0 absences
    [1, 3], # Student 3: 1 hour study, 3 absences
    [4, 0], # Student 4: 4 hours study, 0 absences
    [1, 2], # Student 5: 1 hour study, 2 absences
    [3, 1], # Student 6: 3 hours study, 1 absence
    [0, 4], # Student 7: 0 hours study, 4 absences
    [5, 0], # Student 8: 5 hours study, 0 absences
    [2, 2], # Student 9: 2 hours study, 2 absences
    [4, 1] # Student 10: 4 hours study, 1 absence
])

# Target variable: Pass (1) or Fail (0)
y = np.array([
    1, # Student 1: Pass
    1, # Student 2: Pass
    0, # Student 3: Fail
    1, # Student 4: Pass
    0, # Student 5: Fail
    1, # Student 6: Pass
    0, # Student 7: Fail
    1, # Student 8: Pass
    0, # Student 9: Fail
    1 # Student 10: Pass
])

print("Dataset created:")
print(f"Features (X - Study Hours, Absences):\n", X)
```

58

Figure 10.2 — Sample dataset (10 students × 2 features: Study Hours, Absences)

```

... Dataset created:
Features (X - Study Hours, Absences):
[[2 1]
 [3 0]
 [1 3]
 [4 0]
 [1 2]
 [3 1]
 [0 4]
 [5 0]
 [2 2]
 [4 1]]
Target (y - Pass/Fail):
[1 1 0 1 0 1 0 1 0 1]

```

Split Dataset in training and testing

```

▶ x_train, x_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

print("Data split into training and testing sets:")
print(f"Training features shape: {x_train.shape}")
print(f"Testing features shape: {x_test.shape}")
print(f"Training target shape: {y_train.shape}")
print(f"Testing target shape: {y_test.shape}")

... Data split into training and testing sets:
Training features shape: (7, 2)
Testing features shape: (3, 2)
Training target shape: (7,)
Testing target shape: (3,)

```

Train a Naïve Bayes

```

▶ # Create a Gaussian Naïve Bayes classifier
model = GaussianNB()

# Train the model using the training data
model.fit(x_train, y_train)

print("Gaussian Naïve Bayes model trained successfully!")

... Gaussian Naïve Bayes model trained successfully!

```

Figure 10.3 — Dataset split, GaussianNB training and predictions

Predict on test data

```
# Make predictions on the test set
y_pred = model.predict(x_test)

print("Predictions made on the test data.")

... Predictions made on the test data.
```

Evaluate Model Performance

```
print("\n--- Model Evaluation ---")

# 1. Print Predictions vs. Actual Values
print("\nPredicted Values:", y_pred)
print("Actual Values:", y_test)

# 2. Accuracy Score
accuracy = metrics.accuracy_score(y_test, y_pred)
print(f"\nAccuracy Score: {accuracy:.2f}")

# 3. Confusion Matrix
# A confusion matrix shows the number of correct and incorrect predictions
# for each class (Pass and Fail).
# [[True Negatives, False Positives]
#  [False Negatives, True Positives]]
conf_matrix = metrics.confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:\n", conf_matrix)

# 4. Classification Report
# Provides precision, recall, and F1-score for each class.
# Precision: Of all predicted positives, how many were actually positive?
# Recall: Of all actual positives, how many were predicted positive?
# F1 score: Harmonic mean of precision and recall.
class_report = metrics.classification_report(y_test, y_pred)
print("\nClassification Report:\n", class_report)
```

```
...
--- Model Evaluation ---

Predicted Values: [0 1 1]
Actual Values: [0 1 1]

Accuracy Score: 1.00

Confusion Matrix:
[[1 0]
 [0 2]]

Classification Report:
      precision    recall  f1-score   support

     0       1.00      1.00      1.00         1
     1       1.00      1.00      1.00         2

   accuracy          1.00
  macro avg          1.00
weighted avg          1.00
```

Figure 10.4 — Model evaluation: Accuracy 1.00, Confusion Matrix and Classification Report

Lab 11: Support Vector Machine (Credit Card Fraud Detection)

Introduction

Support Vector Machine (SVM) is a powerful supervised machine learning algorithm for classification and regression. It classifies data by finding the best decision boundary — called a *hyperplane* — that separates classes with the maximum possible margin. The data points closest to the boundary are called **Support Vectors**.

Key Concepts

Hyperplane	Decision boundary separating classes. In 2D, it is a line.
Margin	Distance between the hyperplane and the nearest data points from each class. Larger margins improve generalisation.
Support Vectors	Data points nearest to the hyperplane that determine its position and orientation.
Kernel Trick	Technique for non-linearly separable data. Common kernels: Linear, Polynomial, RBF, Sigmoid.

Implementation

Dataset

```
[8] ✓ 0s X_svm = np.array([
    [2, 8], # Pass: High study, high attendance
    [3, 7], # Pass
    [4, 9], # Pass
    [1, 6], # Fail: Low study, medium attendance
    [0, 5], # Fail
    [2, 7.5], # Pass
    [0.5, 4], # Fail
    [3.5, 8.5], # Pass
    [1.5, 5], # Fail
    [5, 9.5] # Pass
])

# Target variable: Pass (1) or Fail (0)
y_svm = np.array([
    1, # Pass
    1, # Pass
    1, # Pass
    0, # Fail
    0, # Fail
    1, # Pass
    0, # Fail
    1, # Pass
    0, # Fail
    1 # Pass
])

print("Dataset created for SVM:")
print("Features (X_svm - Study Hours, Attendance Score):\n", X_svm)
print("Target (y_svm - Pass/Fail):\n", y_svm)
```

```

... Dataset created for SVM:
Features (X_svm - Study Hours, Attendance Score):
[[2.  8. ]
 [3.  7. ]
 [4.  9. ]
 [1.  6. ]
 [0.  5. ]
 [2.  7.5]
 [0.5 4. ]
 [3.5 8.5]
 [1.5 5. ]
 [5.  9.5]]
Target (y_svm - Pass/Fail):
[1 1 1 0 0 1 0 1 0 1]

```

Split Dataset

```

[9] X_train_svm, X_test_svm, y_train_svm, y_test_svm = train_test_split(X_svm, y_svm, test_size=0.3, random
0s
print("Data split into training and testing sets for SVM:")
print(f"Training features shape: {X_train_svm.shape}")
print(f"Testing features shape: {X_test_svm.shape}")
print(f"Training target shape: {y_train_svm.shape}")
print(f"Testing target shape: {y_test_svm.shape}")

... Data split into training and testing sets for SVM:
Training features shape: (7, 2)
Testing features shape: (3, 2)
Training target shape: (7,)
Testing target shape: (3,)
+ Code + Text

```

Train SVM

```

[10] # Create an SVM classifier with a linear kernel
0s svm_model = SVC(kernel='linear', random_state=42)

# Train the model using the training data
svm_model.fit(X_train_svm, y_train_svm)

print("SVM model with linear kernel trained successfully!")

... SVM model with linear kernel trained successfully!

```

Figure 11.2 — Dataset split (7 train / 3 test) and SVM linear kernel training

Predict on Data

```
[11]
✓ Os      # Make predictions on the test set
          y_pred_svm = svm_model.predict(X_test_svm)

          print("Predictions made on the test data.")

✓ Predictions made on the test data.
```

Evaluate

```
[12]
✓ Os      print("\n--- SVM Model Evaluation ---")

          # 1. Print Predictions vs. Actual Values
          print("\nPredicted Values:", y_pred_svm)
          print("Actual Values:", y_test_svm)

          # 2. Accuracy Score
          accuracy_svm = metrics.accuracy_score(y_test_svm, y_pred_svm)
          print(f"\nAccuracy Score: {accuracy_svm:.2f}")

          # 3. Confusion Matrix
          # This shows how many predictions were correct and incorrect for each class.
          # [[True Negatives, False Positives]
          #  [False Negatives, True Positives]]
          conf_matrix_svm = metrics.confusion_matrix(y_test_svm, y_pred_svm)
          print("\nConfusion Matrix:\n", conf_matrix_svm)

          # 4. Classification Report
          # Provides precision, recall, and F1-score for each class (0=Fail, 1=Pass).
          class_report_svm = metrics.classification_report(y_test_svm, y_pred_svm)
          print("\nClassification Report:\n", class_report_svm)
```

```
...
--- SVM Model Evaluation ---

Predicted Values: [0 1 1]
Actual Values: [0 1 1]

Accuracy Score: 1.00

Confusion Matrix:
[[1 0]
 [0 2]]

Classification Report:
              precision    recall  f1-score   support

     0           1.00        1.00        1.00         1
     1           1.00        1.00        1.00         2

   accuracy                   1.00         3
  macro avg           1.00        1.00        1.00         3
 weighted avg           1.00        1.00        1.00         3
```

Figure 11.3 — SVM Model Evaluation: Accuracy 1.00, Precision/Recall/F1 all 1.00

Graph

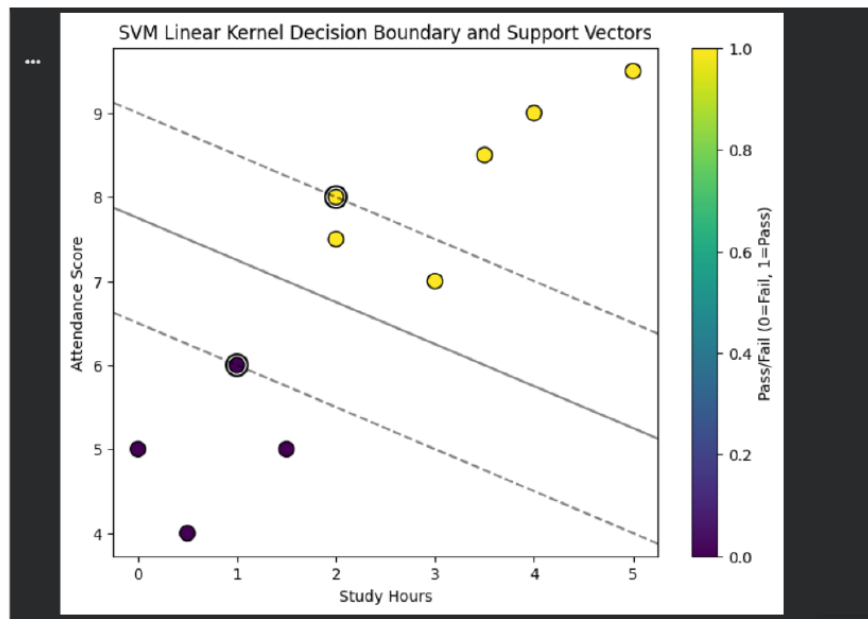


Figure 11.4 — SVM Linear Kernel Decision Boundary and Support Vectors visualisation

Lab 12: Clustering (Customer Segmentation)

Theory

K-Means is an unsupervised machine learning algorithm that divides data into K clusters. A cluster is a group of data points more similar to each other than to points in other clusters. The algorithm minimises the **Within-Cluster Sum of Squares (WCSS)** by repeatedly assigning data points to the nearest centroid and updating centroid positions until convergence.

Algorithm Steps

- Select the number of clusters K.
- Initialise K centroids randomly.
- Assign each data point to the nearest centroid.
- Recalculate centroids as the mean of assigned points.
- Repeat assignment and update steps until convergence.

Applications: customer segmentation, image compression, document grouping, market analysis, recommendation systems, anomaly detection.

Implementation

```
import matplotlib.pyplot as plt
import numpy as np
# 10 data points (2 features)
X = np.array([
    [1, 2],
    [2, 1],
    [1, 1],
    [2, 2],
    [8, 8],
    [9, 8],
    [8, 9],
    [9, 9],
    [5, 6],
    [6, 5]
])
# Plot data
plt.scatter(X[:, 0], X[:, 1])
plt.title("Original Dataset")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```

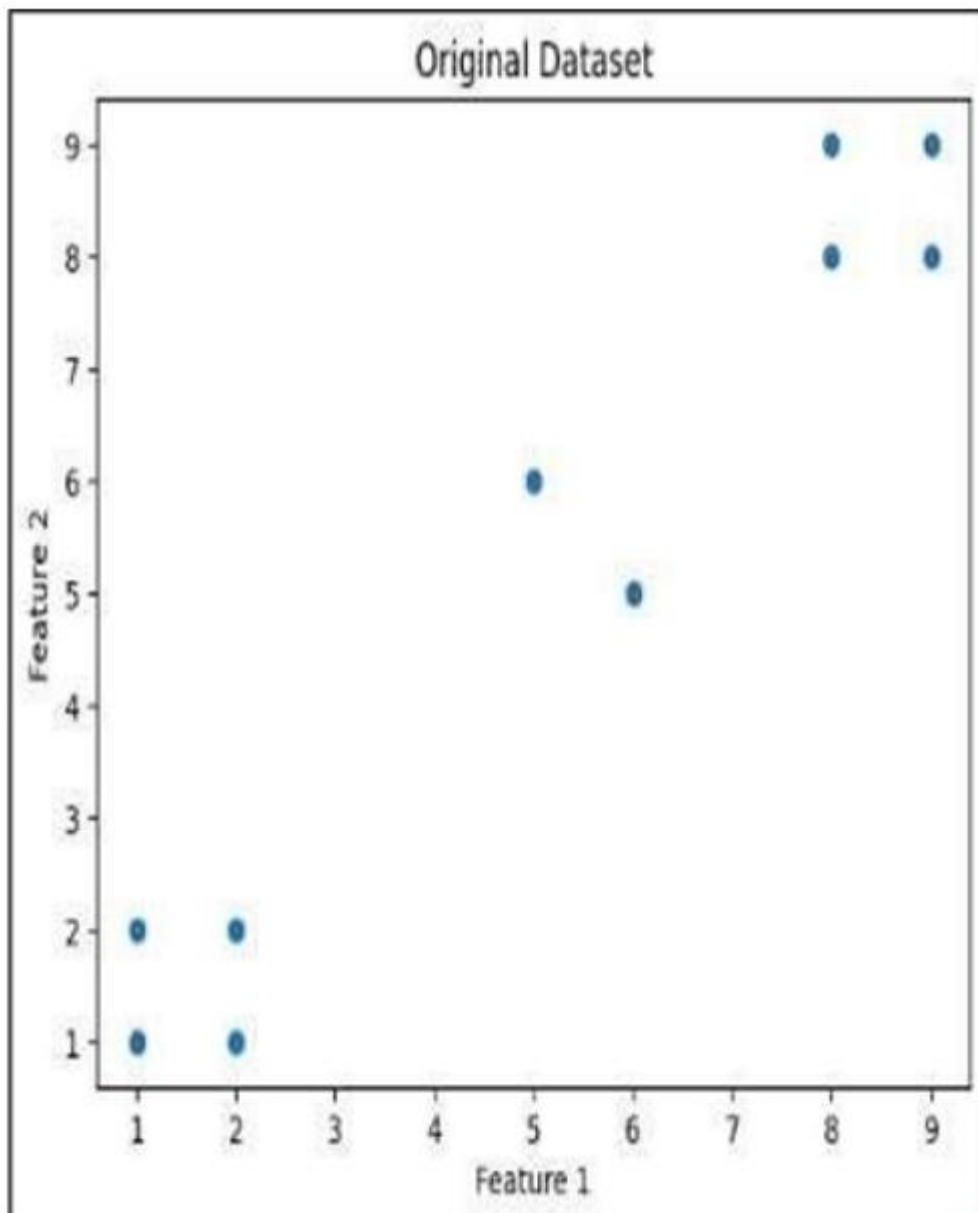


Figure 12.1 — 10-point dataset (2 features) and original scatter plot

```

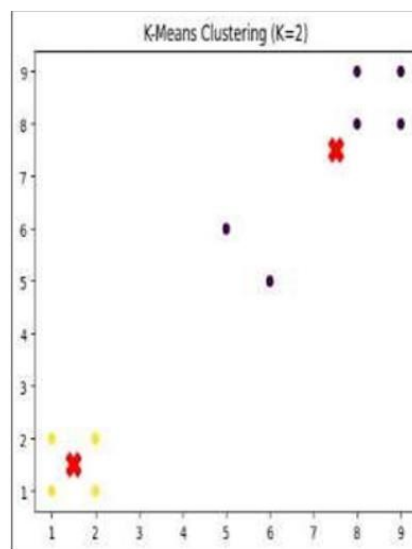
from sklearn.cluster import KMeans
# K = 2
kmeans2 = KMeans(n_clusters=2, random_state=42, n_init=10)
labels2 = kmeans2.fit_predict(X)
# K = 3
kmeans3 = KMeans(n_clusters=3, random_state=42, n_init=10)
labels3 = kmeans3.fit_predict(X)

```

```

plt.scatter(X[:, 0], X[:, 1], c=labels2, cmap='viridis')
plt.scatter(kmeans2.cluster_centers[:, 0],
            kmeans2.cluster_centers[:, 1],
            color='red', marker='X', s=200)
plt.title("K-Means Clustering (K=2)")
plt.show()

```



```

plt.scatter(X[:, 0], X[:, 1], c=labels3, cmap='rainbow')
plt.scatter(kmeans3.cluster_centers[:, 0],
            kmeans3.cluster_centers[:, 1],
            color='black', marker='X', s=200)

plt.title("K-Means Clustering (K=3)")
plt.show()

```

Figure 12.2 — K-Means clustering with K=2 and K=3; centroids marked with X

K-Nearest Neighbours (KNN)

KNN is a supervised classification algorithm that assigns a new data point to the class most common among its K nearest neighbours. It is non-parametric and instance-based — no model is trained; predictions are made directly from stored training examples.

KNN

```
from sklearn.neighbors import KNeighborsClassifier
# Simple labeled dataset
X_train = np.array([
    [1, 1],
    [2, 2],
    [3, 3],
    [6, 6],
    [7, 7],
    [8, 8]
])

y_train = np.array([0, 0, 0, 1, 1, 1])
```

```
# K = 3
knn3 = KNeighborsClassifier(n_neighbors=3)
knn3.fit(X_train, y_train)

# K = 5
knn5 = KNeighborsClassifier(n_neighbors=5)
knn5.fit(X_train, y_train)
```

... **KNeighborsClassifier** ⓘ ⓘ
KNeighborsClassifier()

70

Figure 12.3 — KNN with K=3 and K=5 trained on 6-point dataset; both predict class [0] for point [4,4]

Lab 13: Outlier Detection (Abnormal Banking Transactions)

Detection Methods

Outliers are data points far from most other points. A point is considered an outlier if it has very few nearby neighbours within a distance R. <i>Idea: "Far from others = Outlier"</i>	Isolation Forest is an anomaly detection algorithm. It works by randomly splitting data. Outliers get isolated quickly (short path length). <i>Idea: "Easy to isolate = Outlier"</i>

Common Datasets for Outlier Detection

- Iris dataset — classic benchmark for anomaly detection algorithms.
- Credit Card Fraud dataset — detecting fraudulent transactions.
- Wine Quality dataset — identifying atypical batches.
- Breast Cancer dataset — flagging unusual diagnostic patterns.

Implementation

- Iris dataset
- Credit Card Fraud dataset
- Wine Quality dataset
- Breast Cancer dataset Used to test anomaly detection algorithms in real life.

Idea: “ Easy to isolate = Outlier ”

Paractical Dataset

Common datasets used for outlier detection:

- Iris dataset
- Credit Card Fraud dataset
- Wine Quality dataset
- Breast Cancer dataset Used to test anomaly detection algorithms in real life.

```
[38]  import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.ensemble import IsolationForest
from sklearn.neighbors import NearestNeighbors
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import load_iris
data = load_iris()
X = pd.DataFrame(data.data, columns=data.feature_names)
```

72

Figure 13.1 — Iris dataset loaded; IsolationForest and NearestNeighbors imported

```
 k = 5
nbrs = NearestNeighbors(n_neighbors=k)
nbrs.fit(X)
distances, indices = nbrs.kneighbors(X)
outlier_scores = distances.mean(axis=1)
X['Outlier_Score'] = outlier_scores
threshold = np.percentile(outlier_scores, 95)
X['Outlier'] = outlier_scores > threshold
print(X.head())
```

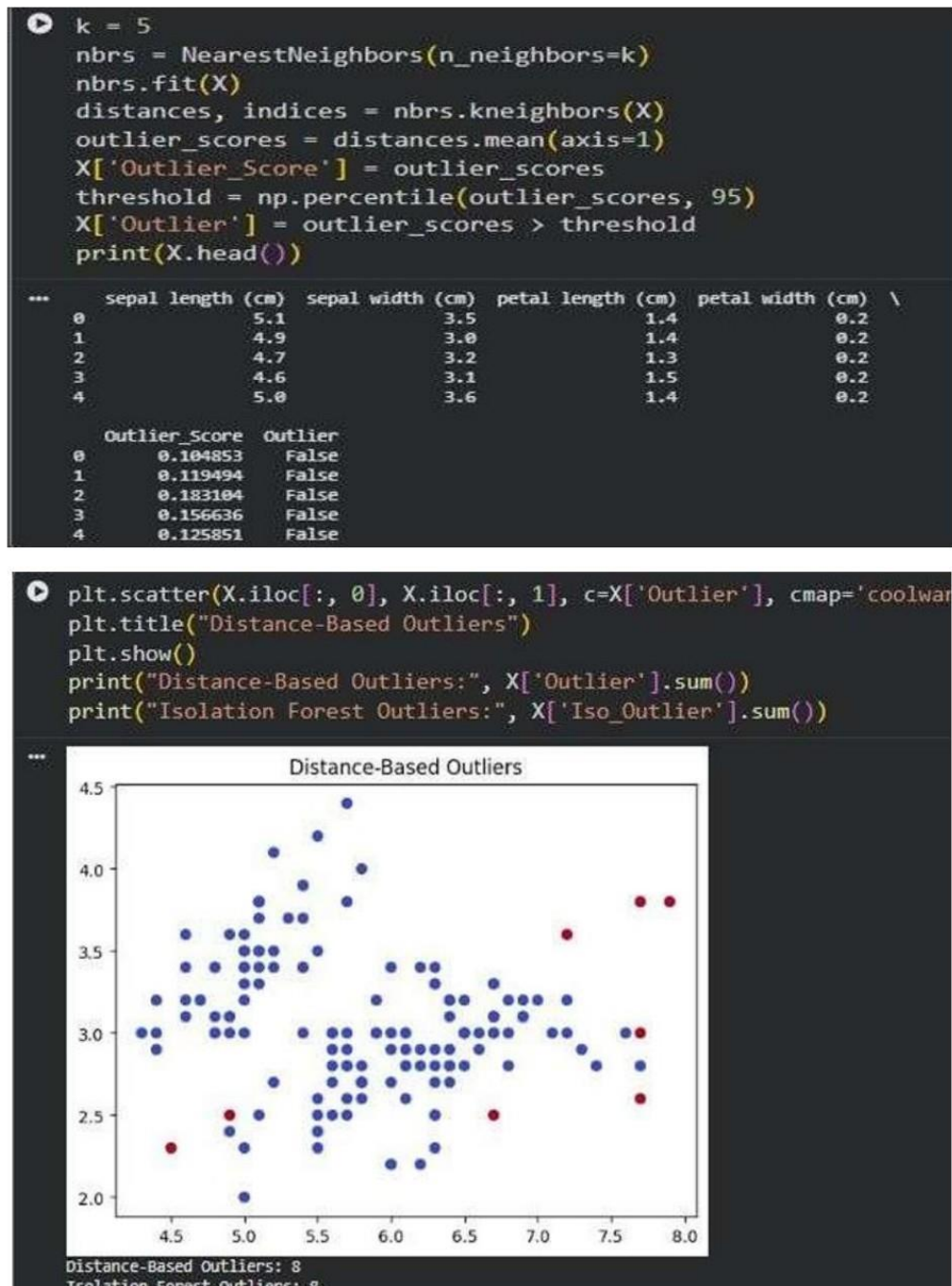


Figure 13.2 — Distance-based outlier scores ($k=5$ neighbours) and scatter plot; 8 outliers detected by both methods

Lab 14: Scraping & Sentiment Analysis (Product Reviews)

Theory

Sentiment Analysis is an NLP technique used to determine the emotional tone of text, automatically classifying content as Positive, Negative, or Neutral. Organisations use it to understand customer opinions from reviews, social media posts, surveys, and feedback.

VADER Sentiment Analyzer (Valence Aware Dictionary and Sentiment Reasoner) is a lexicon and rule-based tool designed for social media content and short texts. It produces four scores — Positive, Negative, Neutral, and Compound. The Compound score summarises overall sentiment on a scale from -1 to +1. Classification: Compound $\geq 0.05 \rightarrow$ Positive; $\leq -0.05 \rightarrow$ Negative; otherwise Neutral.

Applications

- Product review analysis and brand reputation management.
- Social media monitoring and public opinion tracking.
- Customer feedback evaluation and market research.
- Political analysis and business intelligence.

Examples

Review Text	Sentiment
"This mobile phone is excellent"	✓ Positive
"The service was very bad"	✗ Negative
"The product is average"	■ Neutral
"I am happy with the purchase"	✓ Positive
"The delivery was delayed and disappointing"	✗ Negative

Implementation

```
!pip install vaderSentiment
import pandas as pd
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
analyzer = SentimentIntensityAnalyzer()
reviews = [
    "This mobile phone is excellent",
```

"I hate this service." → Negative

"The product is okay." → Neutral

```
!pip install vaderSentiment
import pandas as pd
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
analyzer = SentimentIntensityAnalyzer()
reviews = [
    "This mobile phone is excellent",
    "The service was very bad",
    "The product is average",
    "I am happy with the purchase",
    "The delivery was delayed and disappointing"]
results = []
for review in reviews:
    score = analyzer.polarity_scores(review)
    compound = score['compound']
    if compound >= 0.05:
        sentiment = "Positive"
    elif compound <= -0.05:
        sentiment = "Negative"
    else:
        sentiment = "Neutral"
    results.append([review, sentiment])
df = pd.DataFrame(results, columns=['Review', 'Sentiment'])
print(df)
```

```
results.append([review, sentiment])
df = pd.DataFrame(results, columns=['Review', 'Sentiment'])
print(df)

... Collecting vaderSentiment
  Downloading vaderSentiment-3.3.2-py2.py3-none-any.whl.metadata (572 bytes)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from vaderSentiment) (2.32.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->vaderSentiment) (3.3.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->vaderSentiment) (3.10.1)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->vaderSentiment) (2.2.3)
Requirement already satisfied: certifi<2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->vaderSentiment) (2024.7.4)
  Downloading vaderSentiment-3.3.2-py2.py3-none-any.whl (125 kB)
126.0/126.0 kB 3.6 MB/s eta 0:00:00
Installing collected packages: vaderSentiment
Successfully installed vaderSentiment-3.3.2

   Review Sentiment
0  This mobile phone is excellent Positive
1  The service was very bad Negative
2  The product is average Neutral
3  I am happy with the purchase Positive
4  The delivery was delayed and disappointing Negative
```

75

Figure 14.1 — VADER Sentiment Analyzer applied to 5 product reviews with classification output

Lab 15: Final Capstone (End-to-End Business Solution)

Theoretical Foundation

Objective: Develop a complete data mining solution integrating data preprocessing, sentiment analysis, classification, clustering, and recommendation techniques to extract meaningful insights from customer review data and support business decision-making.

Real-World Scenario

E-commerce companies receive thousands of customer reviews daily. Manual analysis is impractical. Businesses need automated systems that determine customer sentiment, identify customer segments, and recommend products based on purchasing behaviour. This capstone combines multiple data mining techniques into one intelligent analytics system.

Dataset Description

~10,000 customer review records. Attributes: Review ID, Customer ID, Product Name, Product Category, Review Text, Rating, Purchase Date, Customer Age, Customer Location. Combines textual and numerical data across multiple product categories.

Project Workflow

STEP 1 Data Collection

Load the customer review dataset. Import and inspect records. Verify data types.

STEP 2 Data Cleaning

Remove duplicates, handle missing values, standardise text. KPIs: Data Quality Score, Missing Value Reduction %.

STEP 3 Sentiment Analysis

Apply VADER Sentiment Analyzer. Output classes: Positive, Negative, Neutral. KPIs: % Positive Reviews, % Negative Reviews, Average Compound Score.

STEP 4 Classification

Train Decision Tree, Naïve Bayes, and SVM models. Evaluation Metrics: Accuracy, Precision, Recall, F1-Score.

STEP 5 Customer Clustering

Apply K-Means Clustering to group customers by Purchase Frequency, Ratings, and Spending Behaviour. KPIs: Number of Clusters, Cluster Characteristics.

STEP 6 Recommendation

Use association rule mining (Apriori / FP-Growth) with Support, Confidence, and Lift to recommend frequently co-purchased products.

STEP 7 Visualisation

Create charts for Sentiment Distribution, Customer Clusters, Top Products, and Association Rules.

Sample Results

Metric	Result
Positive Reviews	72%
Neutral Reviews	18%
Negative Reviews	10%
Customer Segments	4 distinct clusters identified
Top Recommended Pair	Mobile Cover → Screen Protector
Highest Engagement Category	Electronics

Business Insights

- Customers are generally satisfied with products (72% positive reviews).
- Electronics category generates the highest customer engagement.
- High-spending customers belong primarily to Cluster 2.
- Product bundles can significantly increase sales through cross-selling strategies.

— END OF LAB MANUAL —